

SECTION 11

INSTRUCTION EXECUTION TIMING

This section describes the instruction execution and operations (table searches, etc.) of the MC68030 in terms of external clock cycles. It provides accurate execution and operation timing guidelines but not exact timings for every possible circumstance. This approach is used since exact execution time for an instruction or operation is highly dependent on memory speeds and other variables. The timing numbers presented in this section allow the assembly language programmer or compiler writer to predict actual cache-case and average no-cache-case timings needed to evaluate the performance of the MC68030. Additionally, the timings for exception processing, context switching, and interrupt processing are included so that designers of multi-tasking or real-time systems can predict task switch overhead, maximum interrupt latency, and similar timing parameters.

In this section, instruction and operation times are shown in clock cycles to eliminate clock frequency dependencies.

11.1 PERFORMANCE TRADEOFFS

The MC68030 maximizes average performance at the expense of worst case performance. The time spent executing one instruction can vary from zero to over 100 clocks. Factors affecting the execution time are the preceding and following instructions, the instruction stream alignment, residency of operands and instruction words in the caches, residency of address translations in the address translation cache, and operand alignment.

To increase the average performance of the MC68030, certain tradeoffs were made to increase best case performance and to decrease the occurrence of worst case behavior. For example, burst filling increases performance by prefetching data for later accesses, but it commits the external bus controller and a cache for a longer period.

The MC68030 can overlap data writes with instruction cache reads, data cache reads, and/or microsequencer execution. Instruction cache reads can be overlapped with data cache fills and/or microsequencer activity. Similarly, data cache reads can be overlapped with instruction cache fills and/or micro-

sequencer activity. The execution of an instruction that only accesses on-chip registers can be overlapped entirely with a concurrent data write generated by a previous instruction, if prefetches generated by that instruction are resident in the instruction cache.

11.2 RESOURCE SCHEDULING

Some of the variability in instruction execution timings results from the overlap of resource utilization. The processor can be viewed as consisting of eight independently scheduled resources. Since very little of the scheduling is directly related to instruction boundaries, it is impossible to make accurate estimates of the time required to execute a particular instruction without knowing the complete context within which the instruction is executing. The position of these resources within the MC68030 is shown in Figure 11-1.

11.2.1 Microsequencer

The microsequencer is either executing microinstructions or awaiting completion of accesses that are necessary to continue executing microcode. The bus controller is responsible for all bus activity. The microsequencer controls the bus controller, instruction execution, and internal processor operations such as calculation of effective addresses and setting of condition codes. The microsequencer initiates instruction word prefetches and controls the validation of instruction words in the instruction pipe.

11.2.2 Instruction Pipe

The MC68030 contains a three-word instruction pipe where instruction op-codes are decoded. As shown in Figure 11-1, instruction words (instruction operation words and all extension words) enter the pipe at stage B and proceed to stages C and D. An instruction word is completely decoded when it reaches stage D of the pipe. Each of the pipe stages has a status bit that reflects whether the word in the stage was loaded with data from a bus cycle that was terminated abnormally. Stages of the pipe are only filled in response to specific prefetch requests issued by the microsequencer.

Words are loaded into the instruction pipe from the cache holding register. While the individual stages of the pipe are only 16 bits wide, the cache holding register is 32 bits wide and contains the entire long word. This long word is obtained from the instruction cache or the external bus in response to a prefetch request from the microsequencer. When the microsequencer re-

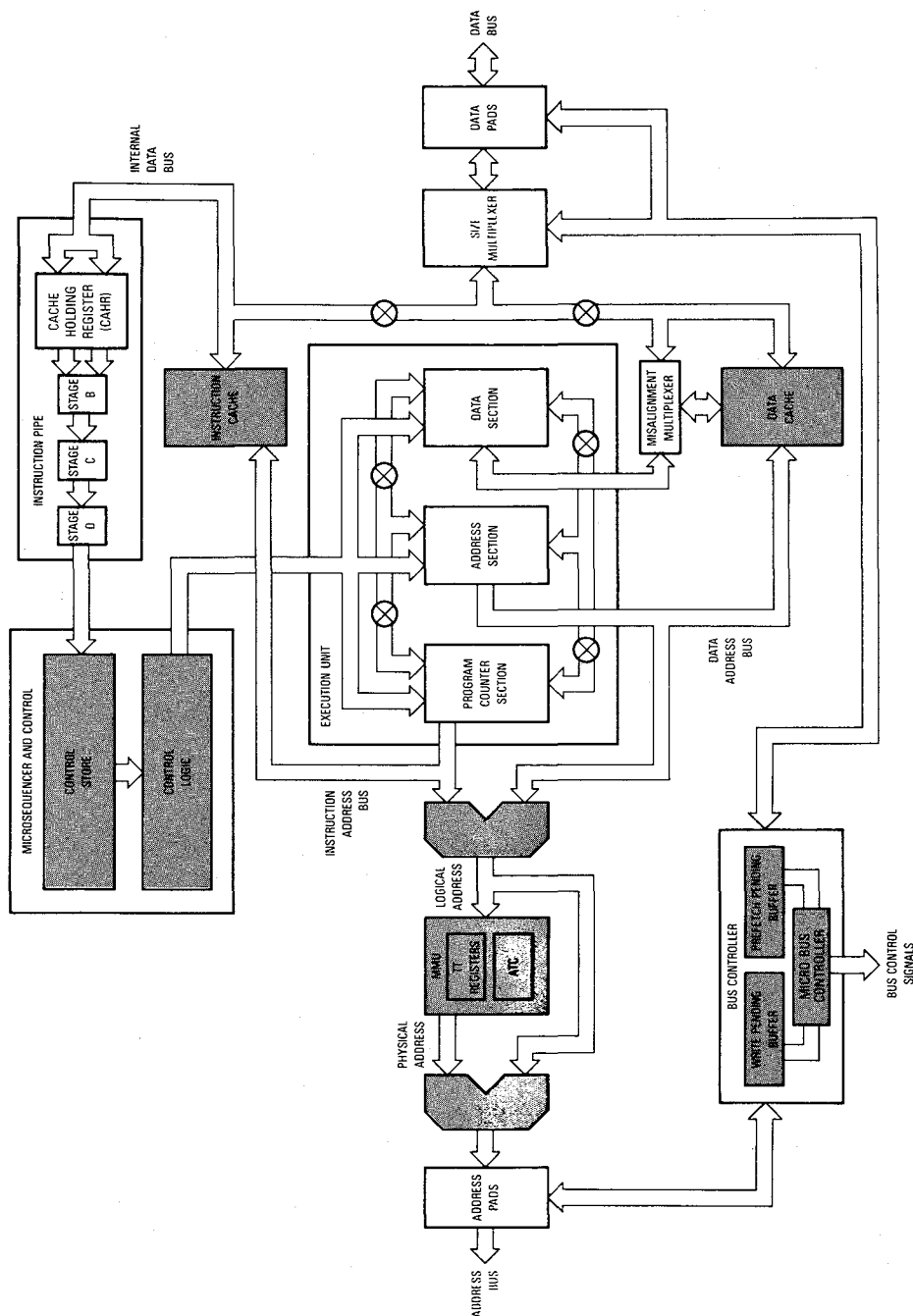


Figure 11-1. Block Diagram — Eight Independent Resources

ests an even-word (long-word aligned) prefetch, the entire long word is accessed from the instruction cache or the external bus and loaded into the cache holding register, and the high-order word is also loaded into stage B of the pipe. The instruction word for the next sequential prefetch can then be accessed directly from the cache holding register, and no external bus cycle or instruction cache access is required. The cache holding register provides instruction words to the pipe, regardless of whether the instruction cache is enabled or disabled.

Prefetch requests are simultaneously submitted to the cache holding register, the instruction cache, and the bus controller. Thus, even if the instruction cache is disabled, an instruction prefetch may hit in the cache holding register and cause an external bus cycle to be aborted.

11.2.3 Instruction Cache

The instruction cache services the instruction prefetch portion of the microsequencer. The prefetch of an instruction that hits in the on-chip instruction cache causes no delay in instruction execution since no external bus activity is required for the prefetch. The instruction cache also interacts with the external bus during instruction cache fills following instruction cache misses.

11.2.4 Data Cache

The data cache services data reads and is updated on data writes. Data operands required by the execution unit that are accessed from the data cache cause no delay in instruction execution due to external bus activity for the data fetch. The data cache also interacts with the external bus during data cache fills following data cache misses.

11.2.5 Bus Controller Resources

Prefetches that miss in the instruction cache cause an external memory cycle to be performed. Similarly, when data reads miss in the on-chip data cache, an external memory cycle is required. The time required for either of these bus cycles may be overlapped with other internal activity.

The bus controller and microsequencer can operate on an instruction concurrently. The bus controller can perform a read or write while the microsequencer controls an effective address calculation or sets the condition

codes. The microsequencer may also request a bus cycle that the bus controller cannot perform immediately. In this case, the bus cycle is queued and the bus controller runs the cycle when the current cycle is complete.

The bus controller consists of the micro bus controller, the instruction fetch pending buffer, and the write pending buffer. These three resources carry out all writes and reads that miss in the on-chip caches.

11.2.5.1 INSTRUCTION FETCH PENDING BUFFER. The instruction prefetch mechanism includes a single long-word instruction fetch pending buffer. Interlocks are provided to prevent this buffer from being overwritten by an instruction prefetch request before a previously requested prefetch is completed.

11.2.5.2 WRITE PENDING BUFFER. The MC68030 incorporates a single write pending buffer, allowing the microsequencer to continue execution after the request for a write cycle proceeds to the bus controller. Interlocks prevent the microsequencer from overwriting this buffer.

11.2.5.3 MICRO BUS CONTROLLER. The micro bus controller performs the bus cycles issued to the bus controller by the rest of the processor. It implements any dynamic bus sizing required and also controls burst operations.

When prefetching instructions from external memory, the micro bus controller utilizes long-word read cycles. The processor reads two words, which may load two instructions at once or two words of a multi-word instruction into the cache holding register (and the instruction cache if it is enabled and not frozen). A special case occurs when prefetch, that, corresponds to an instruction word at an odd-word boundary, is not found in the cache holding register (e.g., due to a branch to an odd-word location) with an instruction cache miss. From a 32-bit memory, the MC68030 reads both the even and odd words associated with the long-word base address in one bus cycle. From an 8- or 16-bit memory, the processor reads the even word before the odd word. Both the even and odd word are loaded into the cache holding register (and the instruction cache if it is enabled and not frozen).

11.2 Memory Management Unit

The MC68030 includes a memory management unit (MMU) that translates logical addresses to physical addresses for external accesses when required. The MMU uses an address translation cache (ATC) to store recently used translations. When the physical address corresponding to a logical address resides in the ATC, the address translation time is completely overlapped with on-chip cache accesses and has no effect on instruction timing.

When the ATC does not contain the translation for a logical address, the processor performs a table search operation to external memory. The amount of time required for a table search depends on the structure of the address translation tree and whether a nonresident portion of the translation tree is required.

The MMU supports demand-paged virtual memory. When a table search terminates with an exception, indicating that the requested instruction or data is not resident, additional time to bring the appropriate page into memory is required. The time required is dependent on the handling routine for the exception.

11.3 INSTRUCTION EXECUTION TIMING CALCULATIONS

The instruction-cache-case timing, overlap, average no-cache-case timing, and actual instruction-cache-case execution time calculations are discussed in the following paragraphs.

11.3.1 Instruction-Cache Case

The instruction-cache-case (CC) time for an instruction is the total number of clock periods required to execute the instruction, provided all the corresponding instruction prefetches are resident in the on-chip instruction cache. All bus cycles are assumed to take two clock periods. The instruction-cache-case time does not assume any overlap with other instructions nor does it take into account hits in the on-chip data cache. The overall instruction-cache-case time for some instructions is divided into the instruction-cache-case time for the required effective address calculation (CC_{ea}) and the instruction-cache-case time for the remainder of the operation (CC_{op}). The instruction-cache-case times for all instructions and addressing modes are listed in the tables of **11.6 INSTRUCTION TIMING TABLES**.

11.3.2 Overlap and Best Case

Overlap is the time, measured in clock periods, that an instruction executes concurrently with the previous instruction. In Figure 11-2, a portion of instructions A and B execute simultaneously. The overlap time decreases the overall execution time for the two instructions. Similarly, an overlap period between instructions B and C reduces the overall execution time of these two instructions.

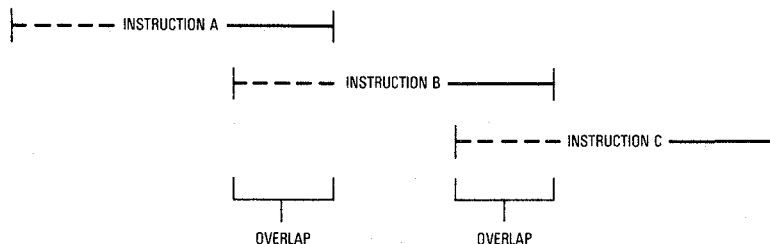


Figure 11-2. Simultaneous Instruction Execution

Each instruction contributes to the total overlap time. As shown in Figure 11-2, a portion of time at the beginning of the execution of instruction B can overlap the end of the execution time of instruction A. This time period is called the head of instruction B. The portion of time at the end of instruction A that can overlap the beginning of instruction B is called the tail of instruction A. The total overlap time between instructions A and B consists of the lesser of the tail of instruction A or the head of instruction B. Refer to the instruction timing tables in **11.6 INSTRUCTION TIMING TABLES** for head and tail times.

Figure 11-3 shows the timing relationship of the factors that comprise the instruction-cache case time for either an effective address calculation (CCea) or for an operation (CCop). In Figure 11-12, the best case execution time for instruction B occurs when the instruction-cache-case times for instruction B and instruction A overlap so that the head of instruction B is completely overlapped with the tail of instruction A.

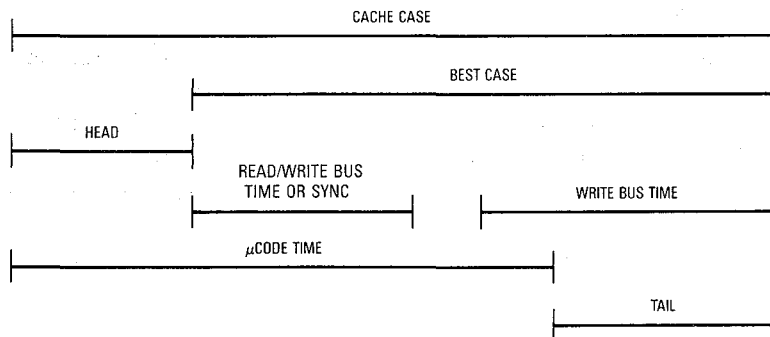


Figure 11-3. Derivation of Instruction Overlap Time

The nature of the instruction overlap and the fact that the heads of some instructions equal the total instruction-cache-case time for those instructions makes a zero net execution time possible. The execution time of an instruction is completely absorbed by overlap with the previous instruction.

11.3.3 Average No-Cache Case

The average no-cache-case (NCC) time for an instruction takes into account the time required for the microcode to execute plus the time required for all external bus activity. This time is calculated assuming both caches miss and the associated instruction prefetches require one external bus cycle per two instruction prefetches. Refer to **11.2.2 Instruction Pipe**. The average no-cache-case time also assumes no overlap. *All bus cycles are assumed to take two clock periods.* Average no-cache-case times for instructions and effective address calculations are listed in **11.6 INSTRUCTION TIMING TABLES**. *Because the no-cache-case times assume no overlap, the head and tail values listed in these tables do not apply to the no-cache-case values.*

Since the actual no-cache-case time depends on the alignment of prefetches associated with an instruction, both alignment cases were considered, and the value shown in the table is the average of the odd-word-aligned case and the even-word-aligned case (rounded up to an integral number of clocks). Similarly, the number of prefetch bus cycles is the average of these two cases rounded up to an integral number of bus cycles.

The effect of instruction alignment on timing is illustrated by the following example. The assumptions referred to in **11.6 INSTRUCTION TIMING TABLES** apply. Both the data cache and instruction cache miss on all accesses.

- Instruction**
1. MOVE.L (d₁₆,An,Dn),Dn
 2. CMPI.W #(data).W,(d₁₆,An)

The instruction stream is positioned with even alignment in 32-bit memory as:

Address	n	MOVE	EA Ext
	n + 4	d ₁₆	CMPI
	n + 8	\$(data.W)	d ₁₆
	n + 12	...	...

Figure 11-4 shows processor activity for even alignment of the given instruction stream. It shows the activity of the external bus, the bus controller, and the sequencer.

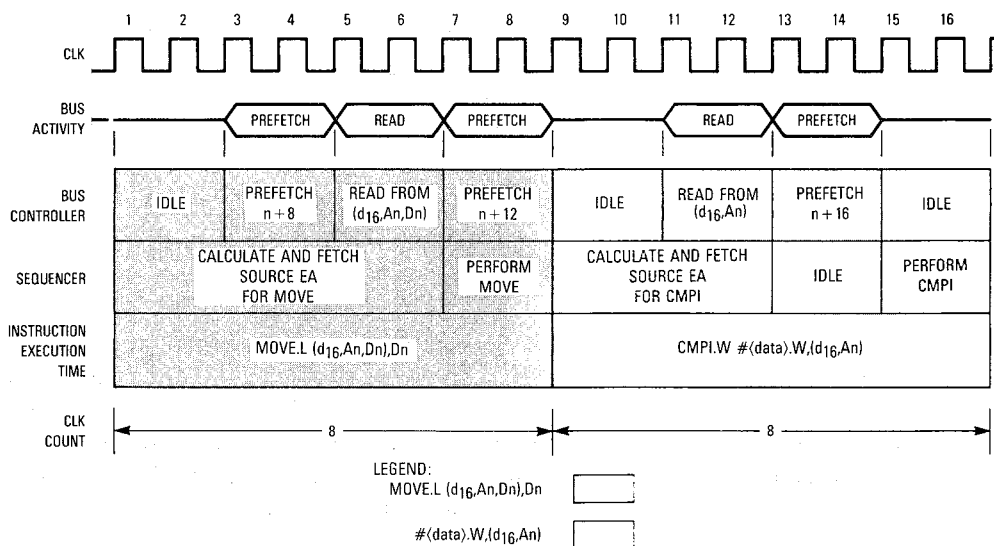


Figure 11-4. Processor Activity — Even Alignment

Figure 11-5 shows processor activity for odd alignment. The instruction stream is positioned in 32-bit memory as:

Address	n	...	MOVE
	n + 4	EA Ext	d ₁₆
	n + 8	CMPI	#{data.W}
	n + 12	d ₁₆	...

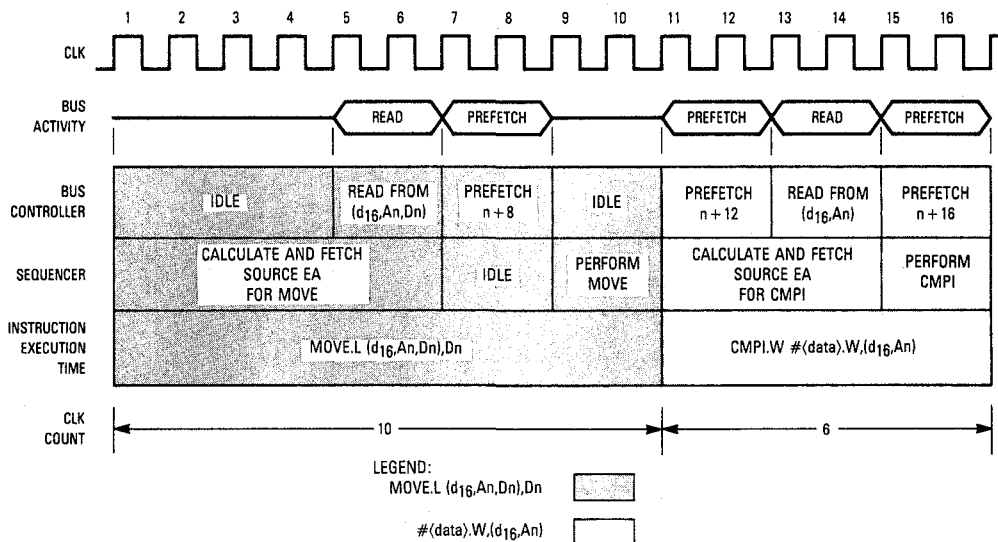


Figure 11-5. Processor Activity — Odd Alignment

Comparing the two alignments, the execution time of the MOVE instruction is eight clocks for even alignment and 10 clocks for odd alignment, an average of nine clocks. Referring to the table in **11.6.6 MOVE Instruction** and the table in **11.6.1 Fetch Effective Address (fea)**, the average no-cache-case time is 2 + 7 = 9 clocks. A similar calculation can be made of the CMPI instruction, which has an average no-cache-case time of seven clocks.

The average no-cache-case timing rather than the maximum no-cache-case timing gives a closer approximation of the actual timing of an instruction stream in many cases. The total execution time of the two instructions in the previous example is 16 clocks for both even and odd alignment. Adding the

average no-cache-case timing of the given instructions also gives 16 clocks ($9 + 7 = 16$ clocks). It should be noted again that the no-cache-case time assumes no overlap. Therefore, the actual execution time of an instruction stream may be less than that given by adding the no-cache-case times. To factor in the effect of wait states for the no-cache case, refer to **11.5 EFFECT OF WAIT STATES**.

11.3.4 Actual Instruction-Cache-Case Execution Time Calculations

The overall execution time for an instruction may depend on the overlap with the previous and following instructions. Therefore, to calculate instruction execution time estimations, the entire code sequence to be evaluated must be analyzed as a whole. To derive the actual instruction-cache-case execution times for an instruction sequence (under the assumptions listed in **11.6 INSTRUCTION TIMING TABLES**), the instruction-cache-case times listed in the tables must be used, and the proper overlap must be subtracted for the entire sequence. The formula for this calculation is:

$$CC_1 + [CC_2 - \min(H_2, T_1)] + [(CC_3 - \min(H_3, T_2))] + \dots \quad (11-1)$$

where:

CC_n is the instruction-cache-case time for an instruction,

T_n is the tail time for an instruction,

H_n is the head time for an instruction, and

$\min(a,b)$ is the minimum of parameters a and b .

The instruction-cache-case time for most instructions is composed of the instruction-cache-case time for the effective address calculation (CC_{ea}) overlapped with the instruction-cache-case time for the operation (CC_{op}). The more specific formula is:

$$CC_{ea1} + [CC_{op1} - \min(H_{op1}, T_{ea1})] + [CC_{ea2} - \min(H_{ea2}, T_{op1})] + [CC_{op2} - \min(H_{op2}, T_{ea2})] + [CC_{ea3} - \min(H_{ea3}, T_{op2})] + \dots \quad (11-2)$$

where:

CC_{ea_n} is the effective address time for the instruction-cache case,

CC_{op_n} is the instruction-cache-case time for the operation portion of an instruction,

T_{ea_n} is the tail time for the effective address of an instruction,

H_{op_n} is the head time for the operation portion of an instruction,

T_{op_n} is the tail time for the operation portion of an instruction,

H_{ea_n} is the head time for the effective address of an instruction, and

$\min(a,b)$ is the minimum of parameters a and b .

The instructions that require the instruction-cache case, head, and tail of an effective address (CCea, Hea, and Tea) to be overlapped with CCop, Hop, and Top are footnoted in **11.6 INSTRUCTION TIMING TABLES**.

The actual instruction-cache-case execution time for a stream of instructions can be computed using Equation (11-1) or the general Equation (11-2). Equation (11-1) is used unless the instruction-cache case, head, and tail of an effective address are required.

An example using a series of instructions that require Equation (11-1) to calculate the instruction-cache-case execution time follows. The assumptions referred to in **11.6 INSTRUCTION TIMING TABLES** apply.

Instruction		
1.	ADD.L	A1,D1
2.	SUBA.L	D1,A2

Referring to the timing table in **11.6.8 Arithmetic/Logical Instructions**, the head, tail, and instruction-cache-case (CC) times for ADD.L A1,D1 and SUBA.L D1,A2 are found. There is no footnote directing the user to add an effective address time for either instruction. Since both of the instructions use register operands only, there is no need to add effective address calculation times. Therefore, the general Equation (11-1) can be used for both.

	Head	Tail	CC
1. ADD.L A1,D1	2	<u>0</u>	2
2. SUBA.L D1,A2	<u>4</u>	0	4

NOTE

The underlined numbers show the typical pattern for the comparison of head and tail in the following equation.

The following computations use Equation (11-1):

$$\begin{aligned}
 \text{Execution Time} &= CC_1 + [CC_2 - \min(H_2, T_1)] \\
 &= 2 + [4 - \min(4, 0)] \\
 &= 2 + [4 - 0] \\
 &= 6 \text{ clocks}
 \end{aligned}$$

Instructions that require the addition of an effective address calculation time from an appropriate table use the general Equation (11-2) to calculate the actual CC time. The CCea, Hea, and Tea values must be extracted from the appropriate effective address table (either fetch effective address, fetch im-

mediate effective address, calculate effective address, calculate immediate effective address, or jump effective address) as indicated and included in Equation (11-2). All of the following instructions (except the last) require general Equation (11-2). The last instruction uses Equation (11-1).

Instruction		
1.	ADD.L	– (A1),D1
2.	AND.L	D1,([A2])
3.	MOVE.L	(A6),(8,A1)
4.	TAS	(A3) +
5.	NEG	D3

Using the appropriate operation and effective address tables from **11.6 INSTRUCTION TIMING TABLES**:

	Head	Tail	CC
1. ADD.L – (A1),D1			
Fetch Effective Address (fea) – (An)	2	2	4
ADD EA,Dn	0	0	2
2. AND.L D1,([A2])			
fea ([B])	4	0	10
AND Dn,EA	0	1	3
3. MOVE.L (A6),(8,A1)			
fea (An)	1	1	3
MOVE Source,(d16,An)	2	0	4
4. TAS (A3) +			
Calculate Effective Address (cea) (An) +	0	0	2
TAS Mem	3	0	12
5. NEG D3	2	0	2

The following calculations use Equations (11-1) and (11-2):

$$\begin{aligned}
 \text{Execution Time} &= \text{CCea}_1 + [\text{CCop}_1 - \min(\text{Hop}_1, \text{Tea}_1)] - [\text{CCea}_2 - \min(\text{Hea}_2, \text{Top}_1)] + \\
 &\quad [\text{CCop}_2 - \min(\text{Hop}_2, \text{Tea}_2)] + [\text{CCea}_3 - \min(\text{Hea}_3, \text{Top}_2)] + \\
 &\quad [\text{CCop}_3 - \min(\text{Hop}_3, \text{Tea}_3)] + [\text{CCea}_4 - \min(\text{Hop}_4, \text{Top}_3)] + \\
 &\quad [\text{CCop}_4 - \min(\text{Hop}_4, \text{Top}_3)] - [\text{CCop}_5 - \min(\text{Hop}_5, \text{Top}_4)] \\
 &= 4 + [2 - \min(0,2)] + [10 - \min(4,0)] - [3 - \min(0,0)] + [3 - \min(1,1)] + \\
 &\quad [4 - \min(2,1)] + [2 - \min(0,0)] + [12 - \min(3,0)] + [2 - \min(2,0)] \\
 &= 4 + 2 + 10 + 3 + 2 + 3 + 2 + 12 + 2 \\
 &= 40 \text{ clock periods}
 \end{aligned}$$

Notice that the last instruction did not require the general Equation (11-2) since there were no effective address (ea) additions. Therefore, Equation (11-1) is used:

$$CCop_5 - \min(Hop_5, Top_4)$$

When using the fetch immediate effective address (fiea) or the calculate immediate effective address (ciea) tables, the size of the data is significant in the timing calculations. For each effective address, a line is listed for word data, #<data>.W, and for long data, #<data>.L.

The total head of some effective address types extends through the effective address calculation and includes the head of the operation. These effective address calculations are marked in the head column as follows:

$$X + \text{op head}$$

where:

X is the head of the effective address alone.

An example using the fiea table and the X + op head notation is:

Instruction			
1.	EORI.W	#\$400, -(A1)	
2.	ADDI.L	#\$6000FF,D1	
	Head	Tail	CC
1. EORI.W #\$400, -(A1)			
fiea #<data>.W, -(An)	2	2	4
EORI #<data>,Mem	0	1	3
2. ADDI.L #\$6000FF,D1			
fiea #<data>.L,D1	4 + op head	0	4
	6	0	4
ADDI #<data>,Dn	2(op head)	0	2

The following calculations use the general Equation (11-2):

$$\begin{aligned}
 \text{Execution Time} &= CCea_1 + [CCop_1 - \min(Hop_1, Tea_1)] + [CCea_2 - \min(Hea_2, Top_1)] + \\
 &\quad [CCop_2 - \min(Hop_2, Tea_2)] \\
 &= 4 + [3 - \min(0, 2)] + [4 - \min(6, 1)] + [2 - \min(2, 0)] \\
 &= 4 + 3 + 3 + 2 \\
 &= 12 \text{ clock periods}
 \end{aligned}$$

Note that for the head of fiea #<data>.L,D1, 4+op head, the resulting head of 6 is larger than the instruction-cache-case time of the fetch. A negative number for the execution time of that portion could result (e.g., $4 - \min(6,6) = -2$). This result would produce the correct execution time since the fetch was completely overlapped and the operation was partially overlapped by the same tail. No changes in the calculation for the operation execution time are required.

Many two-word instructions (e.g., MULU.L, DIV.L, BFSET, etc.) include the fetch immediate effective address (fiea) time or the calculate immediate effective address (ciea) time in the execution time calculation. The timing for immediate data of word length (#<data>.W) is used for these calculations. If the instruction has a source and a destination, the source EA is used for the table lookup. If the instruction is single operand, the effective address of that operand is used.

The following example includes multi-word instructions that refer to the fetch immediate effective address and calculate immediate effective address tables in **11.6 INSTRUCTION TIMING TABLES**.

		Instruction		
1.	MULU.L (D7),D1:D2			
	fiea #<data>.W,Dn	2 + op head	0	2
		4	0	2
	MUL.L EA, Dn	2(op head)	0	44
2.	BFCLR \$6000{0:8}			
	fiea #<data>.W,\$XXX.W	4	2	6
	BFCLR Mem(<5 bytes)	6	0	14
3.	DIVS.L #\$10000,D3:D4			
	fiea #<data>.W,#<data>.L	6 – op head	0	6
		6	0	6
	DIVS.L EA,Dn	0(op head)	0	90

Use the general Equation (11-2) to compute:

$$\begin{aligned}
 \text{Execution Time} &= \text{CCea}_1 + [\text{CCop}_1 - \min(\text{Hop}_1, \text{Tea}_1)] + [\text{CCea}_2 - \min(\text{Hea}_2, \text{Top}_1)] + \\
 &\quad [\text{CCop}_2 - \min(\text{Hop}_2, \text{Tea}_2)] + [\text{CCea}_3 - \min(\text{Hea}_3, \text{Top}_2)] + \\
 &\quad [\text{CCop}_3 - \min(\text{Hop}_3, \text{Tea}_3)] \\
 &= 2 + [44 - \min(2, 0)] + [6 - \min(4, 0)] + [14 - \min(6, 2)] + [6 - \min(6, 0)] + \\
 &\quad [90 - \min(0, 0)] \\
 &= 2 + 44 + 6 + 12 + 6 + 90 \\
 &= 160 \text{ clock periods}
 \end{aligned}$$

NOTE

This CC time is a maximum since the times given for the MULU.L and DIVS.L are maximums.

11.4 EFFECT OF DATA CACHE

When the data accesses required by an instruction are in the data cache, reading these operands requires no bus cycles, and the execution time for the instruction may be minimized. Write accesses, however, always require bus cycles because the data cache is a writethrough cache.

The effect of the data cache on operand read accesses can be factored into the actual instruction execution time as follows.

When a data cache hit occurs for the data fetch corresponding to either the fetch effective address table or the fetch immediate effective address table in **11.6 INSTRUCTION TIMING TABLES**, the following rules apply:

11

- 1a. if $\text{Tail}_t = 0$: No change in timing.
- 1b. if $\text{Tail}_t = 1$: $\text{Tail} = \text{Tail}_t - 1$
 $\text{CC} = \text{CC}_t - 1$
- 1c. if $\text{Tail}_t > 1$: $\text{Tail} = \text{Tail}_t - (\text{Tail}_t - 1) = 1$
 $\text{CC} = \text{CC}_t - (\text{Tail}_t - 1)$
where:
 Tail_t and CC_t are the values listed in the tables.
2. If the EA mode is memory indirect (two data reads), the tail and CC time are calculated as for one data read.

NOTE

Data cache hits cannot easily be accounted for in instruction and operation timings that include an operand fetch in the CCop (e.g., BFFFO and CHK2). The effect of a data cache hit on such CCop's has been ignored for computational purposes.

RMC cycles (e.g., TAS and CAS) are forced to miss on data cache reads. Therefore, a data cache hit has no effect on these instructions.

The following example assumes data cache hits. The lines that are corrected for data cache hits are printed in **boldface** type. These lines are used to calculate the instruction-cache-case execution time. References are to the preceding rules.

Instruction		Head	Tail	CC
1.	ADD.L – (A1),D1			
	Fetch Effective Address			
	fea – (An)	2	2 – 1	4 – 1(1/0/0)
	*1c	2	1	3(1/0/0)
	*ADD EA,Dn	0	0	2(0/0/1)
2.	AND.L D1,([A2])			
	*1a & 2 fea ([B])	4	0	10(2/0/0)
	*AND Dn,EA	0	1	3(0/0/1)
3.	MOVE.L (A6),(8,A1)			
	fea (An)	1	1 – 1	3 – 1(1/0/0)
	*1b	1	0	2(1/0/0)
	*MOVE Source, (d16,An)	2	0	4(0/0/1)
4.	TAS (A3) +			
	*Cea (An) +	0	0	2(0/0/0)
	*TAS Mem	0	0	12(1/0/1)

*Corrected for data cache hits.

NOTE

It is helpful to include the number of operand reads and writes along with the number of instruction accesses in the CC column for computing the effect of data cache hits on execution time.

The following computations use the general Equation (11-2):

$$\begin{aligned}
 \text{Execution Time} &= \text{CCea}_1 + [\text{CCop}_1 - \min(\text{Hop}_1, \text{Tea}_1)] + [\text{CCea}_2 - \min(\text{Hea}_2, \text{Top}_1)] + \\
 &\quad [\text{CCop}_2 - \min(\text{Hop}_2, \text{Tea}_2)] + [\text{CCea}_3 - \min(\text{Hea}_3, \text{Top}_2)] + \\
 &\quad [\text{CCop}_3 - \min(\text{Hop}_3, \text{Tea}_3)] + [\text{CCea}_4 - \min(\text{Hea}_4, \text{Top}_3)] + \\
 &\quad [\text{CCop}_4 - \min(\text{Hop}_4, \text{Tea}_4)] \\
 &= 3 + [2 - \min(0,1)] + [10 - \min(4,0)] + [3 - \min(0,0)] + [2 - \min(1,1)] + \\
 &\quad [4 - \min(2,0)] + [2 - \min(0,0)] + [12 - \min(0,0)] \\
 &= 3 + 2 + 10 + 3 + 1 + 4 + 2 + 12 \\
 &= 37 \text{ clock periods}
 \end{aligned}$$

11.5 EFFECT OF WAIT STATES

The constraints of a system design may require the insertion of wait states in memory cycles. When the bus or the memory device requires many wait states, instruction execution time is increased. However, one or two wait states may have little effect on instruction timing. Often the only effect of one or more wait states is to reduce bus idle time.

The effect of wait states on data accesses may be accounted for in the *instruction-cache-case timings*.

To add the effect of wait states on data accesses:

- 1a. For nonmemory indirect effective address timings that include an operand read, add the number of wait states (in clocks) to the tail and instruction-cache-case (CC) times. The head is not affected.
- 1b. For memory indirect effective address timings that use the calculate <ea> tables and have only one data read (for the address fetch), add the number of wait states to the CC time only. The head and tail are not affected.
- 1c. For memory indirect effective address timings (fetch <ea>) that have two data reads (for the address fetch), add the number of wait states for two reads to the CC time. Add the number of wait states for one data read to the tail. The head is not affected.

- 2a. For operation timings that include a data read (e.g., BFFF0 and TAS), add the number of wait states to the CC time only. Neither the head nor the tail are affected.

NOTE

The CC timing and tail of the MOVEM instruction are special cases for both data reads and writes. Equations for both the CC timing and the tail as a function of wait states are footnoted in the table in **11.6.7 Special-Purpose MOVE Instruction**.

- 2b. If the operation has more than one data read, add the total amount of wait states for all reads to the CC time. Neither the head nor the tail are affected. Refer to preceding note.
- 3a. For operation timings that include a data write, the number of wait states is added to the tail and the CC time. The head is not affected. Refer to preceding note.
- 3b. If there is more than one write in the operation, the tail is only increased by the wait states for one write. The CC timing is increased by the total amount of wait states for all writes. Refer to preceding note.

The following example calculates the instruction-cache-case execution time for the specified instruction stream with two wait states (four-clock reads and writes). The lines that are corrected for wait states are printed in boldface type and are used to calculate the instruction execution time. References are to the preceding rules.

	Instruction
1.	MOVE.L (\$800,A2,D3),(A5,D2)
2.	ADD.L D1,([A30,A4])
3.	BFCLR (\$20,A5){1:5} — (<5 bytes)
4.	BFTST (\$10,A3,D3){31:31} — (5 bytes)
5.	MOVEM ([A1,D1]),A1-A4 — 4 registers



Wait States = 2

	Head	Tail	CC
1. MOVE.L (\$800,A2,D3),(A5,D2) fea (d16,An,Xn) *1a	4 4	0+2 2	6+2(1/0/0) 8(1/0/0)
MOVE Source,(B) *3a	4 4	0+2 2	8+2(0/0/1) 10(0/0/1)
2. ADD.L D1,([\$30,A4]) fea ([d16,B]) *1c	4 4	0+2 2	12+4(2/0/0) 16(2/0/0)
ADD Dn,EA *3a	0 0	1+2 3	3+2(0/0/1) 5(0/0/1)
3. BFCLR (\$20,A5){1:5} *ciea #<data>.W,(d16,An) Single EA Format	10	0	4(0/0/0)
BFCLR Mem (< 5 bytes) *2a & 3a	6 6	0+2 2	14+4(1/0/1) 18(1/0/1)
4. BFTST (\$10,A3,D3){31:31} *ciea (d16,An,Xn)	14	0	8(0/0/0)
BFTST Mem (5 bytes) *2b	6 6	0 0	14+4(2/0/0) 18(2/0/0)
5. MOVEM ([A1,D1]),A1-A4 ciea ([B]) *1b	6 6	0 0	12+2(1/0/0) 14(1/0/0)
MOVEM EA,RL *2a & 2b	2 2	0 0	24+0(4/0/0) 24(4/0/0)

*Corrected for wait states.

NOTE

It is helpful to include the number of operand read and writes along with the number of instruction accesses in the CC column for computing the effect of wait states on execution time.



Using the general Equation (11-2), calculate as follows:

$$\begin{aligned}
 \text{Execution Time} &= \text{CCea}_1 + [\text{CCop}_1 - \min(\text{Hop}_1, \text{Tea}_1)] + [\text{CCea}_2 - \min(\text{Hea}_2, \text{Top}_1)] + \\
 &\quad [\text{CCop}_2 - \min(\text{Hop}_2, \text{Tea}_2)] - [\text{CCea}_3 - \min(\text{Hea}_3, \text{Top}_2)] + \\
 &\quad [\text{CCop}_3 - \min(\text{Hop}_3, \text{Tea}_3)] - [\text{CCea}_4 - \min(\text{Hea}_4, \text{Top}_3)] + \\
 &\quad [\text{CCop}_4 - \min(\text{Hop}_4, \text{Tea}_4)] - [\text{CCea}_5 - \min(\text{Hea}_5, \text{Top}_4)] + \\
 &\quad [\text{CCop}_5 - \min(\text{Hop}_5, \text{Tea}_5)] \\
 &= 8 + [10 - \min(4, 2)] - [16 - \min(4, 2)] + \\
 &\quad [5 - \min(0, 2)] + [4 - \min(10, 3)] + [18 - \min(6, 0)] + [8 - \min(14, 2)] + \\
 &\quad [18 - \min(6, 0)] - [14 - \min(6, 0)] + \\
 &\quad [24 - \min(2, 0)] \\
 &= 8 + 8 + 14 + 5 + 1 - 18 - 6 + 18 + 14 + 24 \\
 &= 116 \text{ clock periods}
 \end{aligned}$$

The next example is the data cache hit example from **11.4 EFFECT OF DATA CACHE** with two wait states per cycle (four-clock read/write). Hits in the data cache and instruction cache are assumed. Three lines are shown for each timing. The first is the timing from the appropriate table. The second is the timing adjusted for a data cache hit. The third *adds wait states only to write operations*, since the read operations hit in the cache and cause no delay. The third line for each timing is used to calculate the instruction cache execution time; it is shown in boldface type.

	Instruction	
1.	ADD.L	— (A1), D1
2.	AND.L	D1, ([A2])
3.	MOVE.L	(A6), (8, A1)
4.	TAS	(A3) ÷

	Head	Tail	CC
1. ADD.L -(A1),D1			
fea -(An)	2	2	4(1/0/0)
*	2	1	3(1/0/0)
**	2	1	3(1/0/0)
ADD.L EA,Dn	0	0	2(0/1/0)
*	0	0	2(0/1/0)
**	0	0	2(0/1/0)
2. AND.L D1,([A1])			
fea ([B])	4	0	10(1/0/0)
*	4	0	10(1/0/0)
***	4	0	12(1/0/0)
AND Dn,EA	0	1	3(0/0/1)
*	0	1	3(0/0/1)
**	0	3	5(0/0/1)
3. MOVE.L (A6),(8,A1)			
fea (An)	1	1	3(1/0/0)
*	1	0	2(1/0/0)
**	1	0	2(1/0/0)
MOVE Source,(d16,An)	2	0	4(0/0/1)
*	2	0	4(0/0/1)
**	2	2	6(0/0/1)
4. TAS (A3) +			
Cea (An)	0	0	2(0/0/0)
*	0	0	2(0/0/0)
**	0	0	2(0/0/0)
TAS Mem	3	0	12(1/0/1)
*	3	0	12(1/0/1)
**	3	0	14(1/0/1)

NOTES:

*Corrected for data cache hits.

**Corrected for wait states also (only on data writes).

***No data cache hit assumed for address fetch.



Using the general Equation (11-2), calculate as follows:

$$\begin{aligned}
 \text{Execution Time} &= \text{CCea}_1 + [\text{CCop}_1 - \min(\text{Hea}_1, \text{Top}_1)] + [\text{CCea}_2 - \min(\text{Hea}_2, \text{Top}_1)] + \\
 &\quad [\text{CCop}_2 - \min(\text{Hop}_2, \text{Tea}_2)] + [\text{CCea}_3 - \min(\text{Hea}_3, \text{Top}_2)] + \\
 &\quad [\text{CCop}_3 - \min(\text{Hop}_3, \text{Tea}_3)] + [\text{CCea}_4 - \min(\text{Hea}_4, \text{Top}_3)] + \\
 &\quad [\text{CCop}_4 - \min(\text{Hop}_4, \text{Tea}_4)] \\
 &= 3 + [2 - \min(0, 1)]m + [12 - \min(4, 0)] + \\
 &\quad [5 - \min(0, 0)] + [2 - \min(1, 3)] + \\
 &\quad [6 - \min(2, 0)] + [2 - \min(0, 2)] + \\
 &\quad [14 - \min(3, 0)] \\
 &= 3 + 2 + 12 + 5 + 1 + 6 + 2 + 14 \\
 &= 45 \text{ clock periods}
 \end{aligned}$$

A similar analysis can be constructed for the average no-cache case. Since the average no-cache-case time assumes two clock periods per bus cycle (i.e., no wait states), the timing given in the tables does not apply directly to systems with wait states. To approximate the average no-cache-case time for an instruction or effective address with W wait states, use the following formula:

$$\text{NCC} = \text{NCC}_t + (\# \text{ of data reads and writes}) \cdot W + (\text{max. \# of instruction accesses}) \cdot W$$

where:

NCC_t is the no-cache-case timing value from the appropriate table.

The number of data reads, data writes, and maximum instruction accesses are found in the appropriate table.

The average no-cache-case timing obtained from this formula is equal to or greater than the actual no-cache-case timing since the number of instruction accesses used is a maximum (the values in the tables are always rounded up) and no overlap is assumed.

1

11.6 INSTRUCTION TIMING TABLES

All the following assumptions apply to the times shown in the tables in this section:

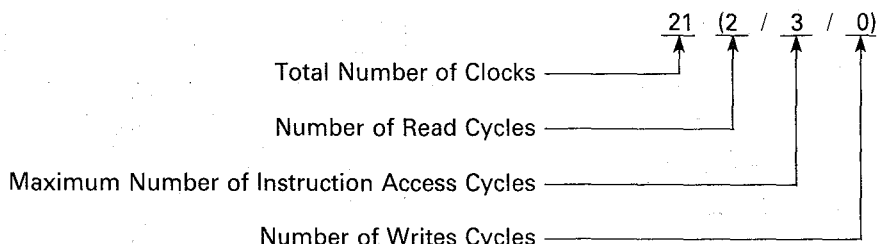
- All memory accesses occur with two-clock bus cycles and no wait states.
- All operands in memory, including the system stack, are long-word aligned.
- A 32-bit bus is used for communications between the MC68030 and system memory.
- The data cache is not enabled.
- No exceptions occur (except as specified).
- Required address translations for all external bus cycles are resident in the address translation cache.

Four values are listed for each instruction and effective address:

1. Head,
2. Tail,
3. Instruction-cache case (CC) when the instruction is in the cache but has no overlap, and
4. Average no-cache case (NCC) when the instruction is not in the cache or the cache is disabled and there is no instruction overlap.

The only instances for which the size of the operand has any effect are the instructions with immediate operands and the ADDA and SUBA instructions. Unless specified otherwise, immediate byte and word operands have identical execution times.

The instruction-cache-case and average no-cache-case columns of the instruction timing tables contain four sets of numbers, three of which are enclosed in parentheses. The outer number is the total number of clocks for the given cache case and instruction. The first number inside the parentheses is the number of operand read cycles performed by the instruction. The second value inside the parentheses is the maximum number of instruction bus cycles performed by the instruction, including all prefetches to keep the instruction pipe filled. Because the second value is the average of the odd-word-aligned case and the even-word-aligned case (rounded up to an integral number of bus cycles), it is always greater than or equal to the actual number of bus cycles (one bus cycle per two instruction prefetches). The third value within the parentheses is the number of write cycles performed by the instruction. One example from the instruction timing table is:



The total numbers of bus-activity clocks and internal clocks (not overlapped by bus activity) of the instruction in this example are derived as follows:

$$(2 \text{ Reads} \cdot 2 \text{ Clocks/Read}) + (3 \text{ Instruction Accesses} \cdot 2 \text{ Clocks/Access}) + (0 \text{ Writes} \cdot 2 \text{ Clocks/Write}) = 10 \text{ Clocks of Bus Activity}$$

$$21 \text{ Total Clocks} - 10 \text{ Bus Activity Clocks} = 11 \text{ Internal Clocks}$$

The example used here is taken from a no-cache-case 'fetch effective address' time. The addressing mode is $([d_{32}, B], I, d_{32})$. The same addressing mode under the instruction-cache-case execution time entry is 18(2/0/0). For the instruction-cache-case execution time, no instruction accesses are required because the cache is enabled and the sequencer does not have to access external memory for the instruction words.

The first five timing tables deal exclusively with fetching and calculating effective addresses and immediate operands. The remaining tables are instruction and operation timings. Some instructions use addressing modes that are not included in the corresponding instruction timings. These cases refer to footnotes that indicate the additional table needed for the timing calculation. All read and write accesses are assumed to take two clock periods.

11.6.1 Fetch Effective Address (fea)

The fetch effective address table indicates the number of clock periods needed for the processor to calculate and fetch the specified effective address. The effective addresses are divided by their formats (refer to **2.5 Effective Address Encoding Summary**). For instruction-cache case and for no-cache case, the total number of clock cycles is outside the parentheses. The number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). The read, prefetch, and write cycles are included in the total clock cycle number.

All timing data assumes two-clock reads and writes.

Address Mode	Head	Tail	I-Cache Case	No-Cache Case
--------------	------	------	--------------	---------------

SINGLE EFFECTIVE ADDRESS INSTRUCTION FORMAT

% Dn	—	—	0(0/0/0)	0(0/0/0)
% An	—	—	0(0/0/0)	0(0/0/0)
(An)	1	1	3(1/0/0)	3(1/0/0)
(An) +	0	1	3(1/0/0)	3(1/0/0)
-(An)	2	2	4(1/0/0)	4(1/0/0)
(d ₁₆ ,An) or (d ₁₆ ,PC)	2	2	4(1/0/0)	4(1/1/0)
(xxx).W	2	2	4(1/0/0)	4(1/1/0)
(xxx).L	1	0	4(1/0/0)	5(1/1/0)
#(data).B	2	0	2(0/0/0)	2(0/1/0)
#(data).W	2	0	2(0/0/0)	2(0/1/0)
#(data).L	4	0	4(0/0/0)	4(0/1/0)

BRIEF FORMAT EXTENSION WORD

(dg,An,Xn) or (dg,PC,Xn)	4	2	6(1/0/0)	6(1/1/0)
--------------------------	---	---	----------	----------

11.6.1 Fetch Effective Address (fea) (Continued)

Address Mode	Head	Tail	I-Cache Case	No-Cache Case
FULL FORMAT EXTENSION WORD(S)				
(d ₁₆ ,An) or (d ₁₆ ,PC)	2	0	6(1/0/0)	7(1/1/0)
(d ₁₆ ,An,Xn) or (d ₁₆ ,PC,Xn)	4	0	6(1/0/0)	7(1/1/0)
((d ₁₆ ,An)) or ((d ₁₆ ,PC))	2	0	10(2/0/0)	10(2/1/0)
((d ₁₆ ,An),Xn) or ((d ₁₆ ,PC),Xn)	2	0	10(2/0/0)	10(2/1/0)
((d ₁₆ ,An),d ₁₆) or ((d ₁₆ ,PC),d ₁₆)	2	0	12(2/0/0)	13(2/2/0)
((d ₁₆ ,An),Xn,d ₁₆) or ((d ₁₆ ,PC),Xn,d ₁₆)	2	0	12(2/0/0)	13(2/2/0)
((d ₁₆ ,An),d ₃₂) or ((d ₁₆ ,PC),d ₃₂)	2	0	12(2/0/0)	14(2/2/0)
((d ₁₆ ,An),Xn,d ₃₂) or ((d ₁₆ ,PC),Xn,d ₃₂)	2	0	12(2/0/0)	14(2/2/0)
(B)	4	0	6(1/0/0)	7(1/1/0)
(d ₁₆ ,B)	4	0	8(1/0/0)	10(1/1/0)
(d ₃₂ ,B)	4	0	12(1/0/0)	13(1/2/0)
((B))	4	0	10(2/0/0)	10(2/1/0)
((B),I)	4	0	10(2/0/0)	10(2/1/0)
((B),d ₁₆)	4	0	12(2/0/0)	13(2/1/0)
((B),I,d ₁₆)	4	0	12(2/0/0)	13(2/1/0)
((B),d ₃₂)	4	0	12(2/0/0)	14(2/2/0)
((B),I,d ₃₂)	4	0	12(2/0/0)	14(2/2/0)
((d ₁₆ ,B))	4	0	12(2/0/0)	13(2/1/0)
((d ₁₆ ,B),I)	4	0	12(2/0/0)	13(2/1/0)
((d ₁₆ ,B),d ₁₆)	4	0	14(2/0/0)	16(2/2/0)
((d ₁₆ ,B),I,d ₁₆)	4	0	14(2/0/0)	16(2/2/0)
((d ₁₆ ,B),d ₃₂)	4	0	14(2/0/0)	17(2/2/0)
((d ₁₆ ,B),I,d ₃₂)	4	0	14(2/0/0)	17(2/2/0)
((d ₃₂ ,B))	4	0	16(2/0/0)	17(2/2/0)
((d ₃₂ ,B),I)	4	0	16(2/0/0)	17(2/2/0)
((d ₃₂ ,B),d ₁₆)	4	0	18(2/0/0)	20(2/2/0)
((d ₃₂ ,B),I,d ₁₆)	4	0	18(2/0/0)	20(2/2/0)
((d ₃₂ ,B),d ₃₂)	4	0	18(2/0/0)	21(2/3/0)
((d ₃₂ ,B),I,d ₃₂)	4	0	18(2/0/0)	21(2/3/0)

B = Base Address; 0, An, PC, Xn, An + Xn, PC + Xn. Form does not affect timing.

I = Index; 0, Xn

% = No clock cycles incurred by effective address fetch.

NOTE: Xn cannot be in B and I at the same time. Scaling and size of Xn do not affect timing.

11.0.2 Fetch Immediate Effective Address (fiea)

The fetch immediate effective address table indicates the number of clock periods needed for the processor to fetch the immediate source operand and to calculate and fetch the specified destination operand. In the case of two-word instructions, this table indicates the number of clock periods needed for the processor to fetch the second word of the instruction and to calculate and fetch the specified source operand or single operand. The effective addresses are divided by their formats (refer to **2.5 Effective Address Encoding Summary**). For instruction-cache case and for no-cache case, the total number of clock cycles is outside the parentheses. The number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). The read, prefetch, and write cycles are included in the total clock cycle number.

All timing data assumes two-clock reads and writes.

Address Mode	Head	Tail	I-Cache Case	No-Cache Case
--------------	------	------	--------------	---------------

SINGLE EFFECTIVE ADDRESS INSTRUCTION FORMAT

% #(data).W,Dn	2 + op head	0	2(0/0/0)	2(0/1/0)
% #(data).L,Dn	4 + op head	0	4(0/0/0)	4(0/1/0)
#{data).W,(An)	1	1	3(1/0/0)	4(1/1/0)
#{data).L,(An)	1	0	4(1/0/0)	5(1/1/0)
#{data).W,(An) +	2	1	5(1/0/0)	5(1/1/0)
#{data).L,(An) +	4	1	7(1/0/0)	7(1/1/0)
#{data).W, - (An)	2	2	4(1/0/0)	4(1/1/0)
#{data).L, - (An)	2	0	4(1/0/0)	5(1/1/0)
#{data).W,(d ₁₆ ,An)	2	0	4(1/0/0)	5(1/1/0)
#{data).L,(d ₁₆ ,An)	4	0	6(1/0/0)	8(1/2/0)
#{data).W,\$XXX.W	4	2	6(1/0/0)	6(1/1/0)
#{data).L,\$XXX.W	6	2	8(1/0/0)	8(1/2/0)
#{data).W,\$XXX.L	3	0	6(1/0/0)	7(1/2/0)
#{data).L,\$XXX.L	5	0	8(1/0/0)	9(1/2/0)
#{data).W,#{data).L	6 + op head	0	6(0/0/0)	6(0/2/0)

BRIEF FORMAT EXTENSION WORD

#{data).W,(dg,An,Xn) or (dg,PC,Xn)	6	2	8(1/0/0)	8(1/2/0)
#{data).L,(dg,An,Xn) or (dg,PC,Xn)	8	2	10(1/0/0)	10(1/2/0)

11.6.2 Fetch Immediate Effective Address (fiea) (Continued)

Address Mode	Head	Tail	I-Cache Case	No-Cache Case
FULL FORMAT EXTENSION WORD(S)				
#(data).W,(d ₁₆ ,An) or (d ₁₆ ,PC)	4	0	8(1/0/0)	9(1/2/0)
#(data).L,(d ₁₆ ,An) or (d ₁₆ ,PC)	6	0	10(1/0/0)	11(1/2/0)
#(data).W,(d ₁₆ ,An,Xn) or (d ₁₆ ,PC,Xn)	6	0	8(1/0/0)	9(1/2/0)
#(data).L,(d ₁₆ ,An,Xn) or (d ₁₆ ,PC,Xn)	8	0	10(1/0/0)	11(1/2/0)
#(data).W,([d ₁₆ ,An]) or ([d ₁₆ ,PC])	4	0	12(2/0/0)	12(2/2/0)
#(data).L,([d ₁₆ ,An]) or ([d ₁₆ ,PC])	6	0	14(2/0/0)	14(2/2/0)
#(data).W,([d ₁₆ ,An],Xn) or ([d ₁₆ ,PC],Xn)	4	0	12(2/0/0)	12(2/2/0)
#(data).L,([d ₁₆ ,An],Xn) or ([d ₁₆ ,PC],Xn)	6	0	14(2/0/0)	14(2/2/0)
#(data).W,([d ₁₆ ,An],d ₁₆) or ([d ₁₆ ,PC],d ₁₆)	4	0	14(2/0/0)	15(2/2/0)
#(data).L,([d ₁₆ ,An],d ₁₆) or ([d ₁₆ ,PC],d ₁₆)	6	0	16(2/0/0)	17(2/3/0)
#(data).W,([d ₁₆ ,An],Xn,d ₁₆) or ([d ₁₆ ,PC],Xn,d ₁₆)	4	0	14(2/0/0)	15(2/2/0)
#(data).L,([d ₁₆ ,An],Xn,d ₁₆) or ([d ₁₆ ,PC],Xn,d ₁₆)	6	0	16(2/0/0)	17(2/3/0)
#(data).W,([d ₁₆ ,An],d ₃₂) or ([d ₁₆ ,PC],d ₃₂)	4	0	14(2/0/0)	16(2/3/0)
#(data).L,([d ₁₆ ,An],d ₃₂) or ([d ₁₆ ,PC],d ₃₂)	6	0	16(2/0/0)	18(2/3/0)
#(data).W,([d ₁₆ ,An],Xn,d ₃₂) or ([d ₁₆ ,PC],Xn,d ₃₂)	4	0	14(2/0/0)	16(2/3/0)
#(data).L,([d ₁₆ ,An],Xn,d ₃₂) or ([d ₁₆ ,PC],Xn,d ₃₂)	6	0	16(2/0/0)	18(2/3/0)
#(data).W,(B)	6	0	8(1/0/0)	9(1/1/0)
#(data).L,(B)	8	0	10(1/0/0)	11(1/2/0)
#(data).W,(d ₁₆ ,B)	6	0	10(1/0/0)	12(1/2/0)
#(data).L,(d ₁₆ ,B)	8	0	12(1/0/0)	14(1/2/0)
#(data).W,(d ₃₂ ,B)	10	0	14(1/0/0)	16(1/2/0)
#(data).L,(d ₃₂ ,B)	12	0	16(1/0/0)	18(1/3/0)
#(data).W,([B])	6	0	12(2/0/0)	12(2/1/0)
#(data).L,([B])	8	0	14(2/0/0)	14(2/2/0)
#(data).W,([B],I)	6	0	12(2/0/0)	12(2 1 0)
#(data).L,([B],I)	8	0	14(2/0/0)	14(2 2 0)
#(data).W,([B],d ₁₆)	6	0	14(2/0/0)	15(2 2 0)
#(data).L,([B],d ₁₆)	8	0	16(2/0/0)	17(2 2 0)
#(data).W,([B],I,d ₁₆)	6	0	14(2/0/0)	15(2 2 0)
#(data).L,([B],I,d ₁₆)	8	0	16(2/0/0)	17(2 2 0)
#(data).W,([B],d ₃₂)	6	0	14(2/0/0)	16(2 2 0)
#(data).L,([B],d ₃₂)	8	0	16(2/0/0)	18(2 3 0)
#(data).W,([B],I,d ₃₂)	6	0	14(2/0/0)	16(2 2 0)
#(data).L,([B],I,d ₃₂)	8	0	16(2/0/0)	18(2 3 0)
#(data).W,([d ₁₆ ,B])	6	0	14(2/0/0)	15(2 2 0)
#(data).L,([d ₁₆ ,B])	8	0	16(2/0/0)	17(2 2 0)
#(data).W,([d ₁₆ ,B],I)	6	0	14(2/0/0)	15(2 2 0)
#(data).L,([d ₁₆ ,B],I)	8	0	16(2/0/0)	17(2 2 0)

11.6.2 Fetch Immediate Effective Address (fiea) (Continued)

Address Mode	Head	Tail	I-Cache Case	No-Cache Case
--------------	------	------	--------------	---------------

FULL FORMAT EXTENSION WORD(S) (CONTINUED)

#(data).W,([d ₁₆ ,B],d ₁₆)	6	0	16(2/0/0)	18(2/2/0)
#(data).L,([d ₁₆ ,B],d ₁₆)	8	0	18(2/0/0)	20(2/3/0)
#(data).W,([d ₁₆ ,B],l,d ₁₆)	6	0	16(2/0/0)	18(2/2/0)
#(data).L,([d ₁₆ ,B],l,d ₁₆)	8	0	18(2/0/0)	20(2/3/0)
#(data).W,([d ₁₆ ,B],d ₃₂)	6	0	16(2/0/0)	19(2/3/0)
#(data).L,([d ₁₆ ,B],d ₃₂)	8	0	18(2/0/0)	21(2/3/0)
#(data).W,([d ₁₆ ,B],l,d ₃₂)	6	0	16(2/0/0)	19(2/3/0)
#(data).L,([d ₁₆ ,B],l,d ₃₂)	8	0	18(2/0/0)	21(2/3/0)
#(data).W,([d ₃₂ ,B])	6	0	18(2/0/0)	19(2/2/0)
#(data).L,([d ₃₂ ,B])	8	0	20(2/0/0)	21(2/3/0)
#(data).W,([d ₃₂ ,B],l)	6	0	18(2/0/0)	19(2/2/0)
#(data).L,([d ₃₂ ,B],l)	8	0	20(2/0/0)	21(2/3/0)
#(data).W,([d ₃₂ ,B],d ₁₆)	6	0	20(2/0/0)	22(2/3/0)
#(data).L,([d ₃₂ ,B],d ₁₆)	8	0	22(2/0/0)	24(2/3/0)
#(data).W,([d ₃₂ ,B],l,d ₁₆)	6	0	20(2/0/0)	22(2/3/0)
#(data).L,([d ₃₂ ,B],l,d ₁₆)	8	0	22(2/0/0)	24(2/3/0)
#(data).W,([d ₃₂ ,B],d ₃₂)	6	0	20(2/0/0)	23(2/3/0)
#(data).L,([d ₃₂ ,B],d ₃₂)	8	0	22(2/0/0)	25(2/4/0)
#(data).W,([d ₃₂ ,B],l,d ₃₂)	6	0	20(2/0/0)	23(2/3/0)
#(data).L,([d ₃₂ ,B],l,d ₃₂)	8	0	22(2/0/0)	25(2/4/0)

B = Base Address: 0, An, PC, Xn, An + Xn, PC + Xn. Form does not affect timing.

l = Index: 0, Xn

% = Total head for fetch immediate effective address timing includes the head time for the operation.

NOTE: Xn cannot be in B and l at the same time. Scaling and size of Xn do not affect timing.

11.6.3 Calculate Effective Address (cea)

The calculate effective address table indicates the number of clock periods needed for the processor to calculate the specified effective address. Fetch time is only included for the first level of indirection on memory indirect addressing modes. The effective addresses are divided by their formats (refer to **2.5 Effective Address Encoding Summary**). For instruction-cache case and for no-cache case, the total number of clock cycles is outside the parentheses. The number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). The read, prefetch, and write cycles are included in the total clock cycle number.

All timing data assumes two-clock reads and writes.

11.6.3 Calculate Effective Address (cea) (Continued)

Address Mode	Head	Tail	I-Cache Case	No-Cache Case
--------------	------	------	--------------	---------------

SINGLE EFFECTIVE ADDRESS INSTRUCTION FORMAT

% Dn	—	—	0(0/0/0)	0(0/0/0)
% An	—	—	0(0/0/0)	0(0/0/0)
(An)	2 + op head	0	2(0/0/0)	2(0/0/0)
(An) +	0	0	2(0/0/0)	2(0/0/0)
– (An)	2 + op head	0	2(0/0/0)	2(0/0/0)
(d ₁₆ ,An) or (d ₁₆ ,PC)	2 + op head	0	2(0/0/0)	2(0/1/0)
(xxx).W	2 + op head	0	2(0/0/0)	2(0/1/0)
(xxx).L	4 + op head	0	4(0/0/0)	4(0/1/0)

BRIEF FORMAT EXTENSION WORD

(dg,An,Xn) or (dg,PC,Xn)	4 + op head	0	4(0/0/0)	4(0/1/0)
--------------------------	-------------	---	----------	----------

FULL FORMAT EXTENSION WORD(S)

(d ₁₆ ,An) or (d ₁₆ ,PC)	2	0	6(0/0/0)	6(0/1/0)
(d ₁₆ ,An,Xn) or (d ₁₆ ,PC,Xn)	6 + op head	0	6(0/0/0)	6(0/1/0)
((d ₁₆ ,An)) or ((d ₁₆ ,PC))	2	0	10(1/0/0)	10(1/1/0)
((d ₁₆ ,An),Xn) or ((d ₁₆ ,PC),Xn)	2	0	10(1/0/0)	10(1/1/0)
((d ₁₆ ,An),d ₁₆) or ((d ₁₆ ,PC),d ₁₆)	2	0	12(1/0/0)	13(1/2/0)
((d ₁₆ ,An),Xn,d ₁₆) or ((d ₁₆ ,PC),Xn,d ₁₆)	2	0	12(1/0/0)	13(1/2/0)
((d ₁₆ ,An),d ₃₂) or ((d ₁₆ ,PC),d ₃₂)	2	0	12(1/0/0)	13(1/2/0)
((d ₁₆ ,An),Xn,d ₃₂) or ((d ₁₆ ,PC),Xn,d ₃₂)	2	0	12(1/0/0)	13(1/2/0)
(B)	6 + op head	0	6(0/0/0)	6(0/1/0)
(d ₁₆ ,B)	4	0	8(0/0/0)	9(0/1/0)
(d ₃₂ ,B)	4	0	12(0/0/0)	12(0/2/0)
((B))	4	0	10(1/0/0)	10(1/1/0)
((B),l)	4	0	10(1/0/0)	10(1/1/0)
((B),d ₁₆)	4	0	12(1/0/0)	13(1/1/0)
((B),l,d ₁₆)	4	0	12(1/0/0)	13(1/1/0)
((B),d ₃₂)	4	0	12(1/0/0)	13(1/2/0)
((B),l,d ₃₂)	4	0	12(2/0/0)	13(1/2/0)
((d ₁₆ ,B))	4	0	12(1/0/0)	13(1/1/0)
((d ₁₆ ,B),l)	4	0	12(1/0/0)	13(1/1/0)
((d ₁₆ ,B),d ₁₆)	4	0	14(1/0/0)	16(1/2/0)
((d ₁₆ ,B),l,d ₁₆)	4	0	14(1/0/0)	16(1/2/0)

11.6.3 Calculate Effective Address (cea) (Continued)

Address Mode	Head	Tail	I-Cache Case	No-Cache Case
--------------	------	------	--------------	---------------

FULL FORMAT EXTENSION WORD(S) (CONTINUED)

$\{[d_{16}, B], d_{32}\}$	4	0	14(1/0/0)	16(1/2/0)
$\{[d_{16}, B], I, d_{32}\}$	4	0	14(1/0/0)	16(1/2/0)
$\{[d_{32}, B]\}$	4	0	16(1/0/0)	17(1/2/0)
$\{[d_{32}, B], I\}$	4	0	16(1/0/0)	17(1/2/0)
$\{[d_{32}, B], d_{16}\}$	4	0	18(1/0/0)	20(1/2/0)
$\{[d_{32}, B], I, d_{16}\}$	4	0	18(1/0/0)	20(1/2/0)
$\{[d_{32}, B], d_{32}\}$	4	0	18(1/0/0)	20(1/3/0)
$\{[d_{32}, B], I, d_{32}\}$	4	0	18(1/0/0)	20(1/3/0)

B = Base address; 0, An, PC, Xn, An + Xn, PC + Xn. Form does not affect timing.

I = Index; 0, Xn

% = No clock cycles incurred by effective address calculation.

NOTE: Xn cannot be in B and I at the same time. Scaling and size of Xn do not affect timing.

11.6.4 Calculate Immediate Effective Address (ciea)

The calculate immediate effective address table indicates the number of clock periods needed for the processor to fetch the immediate source operand and calculate the specified destination effective address. In the case of two-word instructions, this table indicates the number of clock periods needed for the processor to fetch the second word of the instruction and calculate the specified source operand or single operand. Fetch time is only included for the first level of indirection on memory indirect addressing modes. The effective addresses are divided by their formats (refer to **2.5 Effective Address Encoding Summary**). For instruction-cache case and for no-cache case, the total number of clock cycles is outside the parentheses. The number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). The read, prefetch, and write cycles are included in the total clock cycle number.

All timing data assumes two-clock reads and writes.

11.6.4 Calculate Immediate Effective Address (ciea) (Continued)

Address Mode	Head	Tail	I-Cache Case	No-Cache Case
--------------	------	------	--------------	---------------

SINGLE EFFECTIVE ADDRESS INSTRUCTION FORMAT

% #(data).W,Dn	2 + op head	0	2(0/0/0)	2(0/1/0)
% #(data).L,Dn	4 + op head	0	4(0/0/0)	4(0/1/0)
% #(data).W,(An)	2 + op head	0	2(0/0/0)	2(0/1/0)
% #(data).L,(An)	4 + op head	0	4(0/0/0)	4(0/1/0)
%(data).W,(An) +	2	0	4(0/0/0)	4(0/1/0)
%(data).L,(An) +	4	0	6(0/0/0)	6(0/1/0)
% #(data).W, -(An)	2 + op head	0	2(0/0/0)	2(0/1/0)
% #(data).L, -(An)	4 + op head	0	4(0/0/0)	4(0/1/0)
% #(data).W,(d16,An)	4 + op head	0	4(0/0/0)	4(0/1/0)
% #(data).L,(d16,An)	6 + op head	0	6(0/0/0)	7(0/2/0)
% #(data).W,\$XXX.W	4 + op head	0	4(0/0/0)	4(0/1/0)
% #(data).L,\$XXX.W	6 + op head	0	6(0/0/0)	6(0/2/0)
% #(data).W,\$XXX.L	6 + op head	0	6(0/0/0)	6(0/2/0)
% #(data).L,\$XXX.L	8 + op head	0	8(0/0/0)	8(0/2/0)

BRIEF FORMAT EXTENSION WORD

% #(data).W,(dg,An,Xn) or (dg,PC,Xn)	6 + op head	0	6(0/0/0)	6(0/2/0)
% #(data).L,(dg,An,Xn) or (dg,PC,Xn)	8 + op head	0	8(0/0/0)	8(0/2/0)

FULL FORMAT EXTENSION WORD(S)

%(data).W,(d16,An) or (d16,PC)	4	0	8(0/0/0)	8(0/2/0)
%(data).L,(d16,An) or (d16,PC)	6	0	10(0/0/0)	10(0/2/0)
% #(data).W,(d16,An,Xn) or (d16,PC,Xn)	8 + op head	0	8(0/0/0)	8(0/2/0)
% #(data).L,(d16,An,Xn) or (d16,PC,Xn)	10 + op head	0	10(0/0/0)	10(0/2/0)
%(data).W,({d16,An}) or ({d16,PC})	4	0	12(1/0/0)	12(1/2/0)
%(data).L,({d16,An}) or ({d16,PC})	6	0	14(1/0/0)	14(1/1/0)
%(data).W,({d16,An},Xn) or ({d16,PC},Xn)	4	0	12(1/0/0)	12(1/2/0)
%(data).L,({d16,An},Xn) or ({d16,PC},Xn)	6	0	14(1/0/0)	14(1/1/0)
%(data).W,({d16,An},d16) or ({d16,PC},d16)	4	0	14(1/0/0)	15(1/2/0)
%(data).L,({d16,An},d16) or ({d16,PC},d16)	6	0	16(1/0/0)	17(1/3/0)
%(data).W,({d16,An},Xn,d16) or ({d16,PC},Xn,d16)	4	0	14(1/0/0)	15(1/2/0)
%(data).L,({d16,An},Xn,d16) or ({d16,PC},Xn,d16)	6	0	16(1/0/0)	17(1/3/0)
%(data).W,({d16,An},d32) or ({d16,PC},d32)	4	0	14(1/0/0)	16(1/3/0)
%(data).L,({d16,An},d32) or ({d16,PC},d32)	6	0	16(1/0/0)	17(1/3/0)
%(data).W,({d16,An},Xn,d32) or ({d16,PC},Xn,d32)	4	0	14(1/0/0)	15(1/3/0)
%(data).L,({d16,An},Xn,d32) or ({d16,PC},Xn,d32)	6	0	16(1/0/0)	17(1/3/0)
% #(data).W,(B)	8 + op head	0	8(0/0/0)	8(0/1/0)
% #(data).L,(B)	10 + op head	0	10(0/0/0)	10(0/2/0)

11.6.4 Calculate Immediate Effective Address (ciea) (Continued)

Address Mode	Head	Tail	I-Cache Case	No-Cache Case
--------------	------	------	--------------	---------------

FULL FORMAT EXTENSION WORD(S) (CONTINUED)

#(data).W,(d ₁₆ ,B)	6	0	10(0/0/0)	11(0/2/0)
#(data).L,(d ₁₆ ,B)	8	0	12(0/0/0)	13(0/2/0)
#(data).W,(d ₃₂ ,B)	6	0	14(0/0/0)	15(0/2/0)
#(data).L,(d ₃₂ ,B)	8	0	16(0/0/0)	17(0/3/0)
#(data).W,([B])	6	0	12(1/0/0)	12(1/1/0)
#(data).L,([B])	8	0	14(1/0/0)	14(1/2/0)
#(data).W,([B],I)	6	0	12(1/0/0)	12(1/1/0)
#(data).L,([B],I)	8	0	14(1/0/0)	14(1/2/0)
#(data).W,([B],d ₁₆)	6	0	14(1/0/0)	15(1/2/0)
#(data).L,([B],d ₁₆)	8	0	16(1/0/0)	17(1/2/0)
#(data).W,([B],I,d ₁₆)	6	0	14(1/0/0)	15(1/2/0)
#(data).L,([B],I,d ₁₆)	8	0	16(2/0/0)	17(1/2/0)
#(data).W,([B],d ₃₂)	6	0	14(1/0/0)	15(1/2/0)
#(data).L,([B],d ₃₂)	8	0	16(1/0/0)	17(1/3/0)
#(data).W,([B],I,d ₃₂)	6	0	14(1/0/0)	15(1/2/0)
#(data).L,([B],I,d ₃₂)	8	0	16(1/0/0)	17(1/3/0)
#(data).W,([d ₁₆ ,B])	6	0	14(1/0/0)	15(1/2/0)
#(data).L,([d ₁₆ ,B])	8	0	16(1/0/0)	17(1/2/0)
#(data).W,([d ₁₆ ,B],I)	6	0	14(1/0/0)	15(1/2/0)
#(data).L,([d ₁₆ ,B],I)	8	0	16(1/0/0)	17(1/2/0)
#(data).W,([d ₁₆ ,B],d ₁₆)	6	0	16(1/0/0)	18(1/2/0)
#(data).L,([d ₁₆ ,B],d ₁₆)	8	0	18(1/0/0)	20(1/3/0)
#(data).W,([d ₁₆ ,B],I,d ₁₆)	6	0	16(1/0/0)	18(1/2/0)
#(data).L,([d ₁₆ ,B],I,d ₁₆)	8	0	18(1/0/0)	20(1/3/0)
#(data).W,([d ₁₆ ,B],d ₃₂)	6	0	16(1/0/0)	18(1/3/0)
#(data).L,([d ₁₆ ,B],d ₃₂)	8	0	18(1/0/0)	20(1/3/0)
#(data).W,([d ₁₆ ,B],I,d ₃₂)	6	0	16(1/0/0)	18(1/3/0)
#(data).L,([d ₁₆ ,B],I,d ₃₂)	8	0	18(1/0/0)	20(1/3/0)
#(data).W,([d ₃₂ ,B])	6	0	18(1/0/0)	19(1/2/0)
#(data).L,([d ₃₂ ,B])	8	0	20(1/0/0)	21(1/3/0)
#(data).W,([d ₃₂ ,B],I)	6	0	18(1/0/0)	19(1/2/0)
#(data).L,([d ₃₂ ,B],I)	8	0	20(1/0/0)	21(1/3/0)
#(data).W,([d ₃₂ ,B],d ₁₆)	6	0	20(1/0/0)	22(1/3/0)
#(data).L,([d ₃₂ ,B],d ₁₆)	8	0	22(1/0/0)	24(1/3/0)
#(data).W,([d ₃₂ ,B],I,d ₁₆)	6	0	20(1/0/0)	22(1/3/0)
#(data).L,([d ₃₂ ,B],I,d ₁₆)	8	0	22(1/0/0)	24(1/3/0)

11.6.4 Calculate Immediate Effective Address (ciea) (Continued)

Address Mode	Head	Tail	I-Cache Case	No-Cache Case
--------------	------	------	--------------	---------------

FULL FORMAT EXTENSION WORD(S) (CONTINUED)

#(data).W,([d ₃₂ .B],d ₃₂)	6	0	20(1/0/0)	22(1/3/0)
#(data).L,([d ₃₂ .B],d ₃₂)	8	0	22(1/0/0)	24(1/4/0)
#(data).W,([d ₃₂ .B],l,d ₃₂)	6	0	20(1/0/0)	22(1/3/0)
#(data).L,([d ₃₂ .B],l,d ₃₂)	8	0	22(1/0/0)	24(1/4/0)

B = Base address; 0, An, PC, Xn, An + Xn, PC + Xn. Form does not affect timing.

l = Index; 0, Xn

% = Total head for address timing includes the head time for the operation.

NOTE: Xn cannot be in B and l at the same time. Scaling and size of Xn do not affect timing.

11.6.5 Jump Effective Address

The jump effective address table indicates the number of clock periods needed for the processor to calculate the specified effective address for the JMP or JSR instructions. Fetch time is only included for the first level of indirection on memory indirect addressing modes. The effective addresses are divided by their formats (refer to **2.5 Effective Address Encoding Summary**). For instruction-cache case and for no-cache case, the total number of clock cycles is outside the parentheses. The number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). The read, prefetch, and write cycles are included in the total clock cycle number.

All timing data assumes two-clock reads and writes.

Address Mode	Head	Tail	I-Cache Case	No-Cache Case
--------------	------	------	--------------	---------------

SINGLE EFFECTIVE ADDRESS INSTRUCTION FORMAT

% (An)	2 + op head	0	2(0/0/0)	2(0/0/0)
% (d ₁₆ .An)	4 + op head	0	4(0/0/0)	4(0/0/0)
% (xxx).W	2 + op head	0	2(0/0/0)	2(0/0/0)
% (xxx).L	2 + op head	0	2(0/0/0)	2(0/0/0)

BRIEF FORMAT EXTENSION WORD

% (dg.An,Xn) or (dg.PC,Xn)	6 + op head	0	6(0/0/0)	6(0/0/0)
----------------------------	-------------	---	----------	----------

11.6.5 Jump Effective Address (Continued)

Address Mode	Head	Tail	I-Cache Case	No-Cache Case
--------------	------	------	--------------	---------------

FULL FORMAT EXTENSION WORD(S) (CONTINUED)

(d ₁₆ ,An) or (d ₁₆ ,PC)	2	0	6(0/0/0)	6(0/0/0)
% (d ₁₆ ,An,Xn) or (d ₁₆ ,PC,Xn)	6+op head	0	6(0/0/0)	6(0/0/0)
((d ₁₆ ,An)) or ((d ₁₆ ,PC))	2	0	10(1/0/0)	10(1/1/0)
((d ₁₆ ,An),Xn) or ((d ₁₆ ,PC),Xn)	2	0	10(1/0/0)	10(1/1/0)
((d ₁₆ ,An),d ₁₆) or ((d ₁₆ ,PC),d ₁₆)	2	0	12(1/0/0)	12(1/1/0)
((d ₁₆ ,An),Xn,d ₁₆) or ((d ₁₆ ,PC),Xn,d ₁₆)	2	0	12(1/0/0)	12(1/1/0)
((d ₁₆ ,An),d ₃₂) or ((d ₁₆ ,PC),d ₃₂)	2	0	12(1/0/0)	12(1/1/0)
((d ₁₆ ,An),Xn,d ₃₂) or ((d ₁₆ ,PC),Xn,d ₃₂)	2	0	12(1/0/0)	12(1/1/0)
% (B)	6+op head	0	6(0/0/0)	6(0/0/0)
(d ₁₆ ,B)	4	0	8(0/0/0)	9(0/1/0)
(d ₃₂ ,B)	4	0	12(0/0/0)	13(0/1/0)
((B))	4	0	10(1/0/0)	10(1/1/0)
((B),l)	4	0	10(1/0/0)	10(1/1/0)
((B),d ₁₆)	4	0	12(1/0/0)	12(1/1/0)
((B),l,d ₁₆)	4	0	12(1/0/0)	12(1/1/0)
((B),d ₃₂)	4	0	12(1/0/0)	12(1/1/0)
((B),d ₃₂)	4	0	12(1/0/0)	12(1/1/0)
((B),l,d ₃₂)	4	0	12(1/0/0)	12(1/1/0)
((d ₁₆ ,B))	4	0	12(1/0/0)	13(1/1/0)
((d ₁₆ ,B),l)	4	0	12(1/0/0)	13(1/1/0)
((d ₁₆ ,B),d ₁₆)	4	0	14(1/0/0)	15(1/1/0)
((d ₁₆ ,B),l,d ₁₆)	4	0	14(1/0/0)	15(1/1/0)
((d ₁₆ ,B),d ₃₂)	4	0	14(1/0/0)	15(1/1/0)
((d ₁₆ ,B),l,d ₃₂)	4	0	14(1/0/0)	15(1/1/0)
((d ₃₂ ,B))	4	0	16(1/0/0)	17(1/2/0)
((d ₃₂ ,B),l)	4	0	16(1/0/0)	17(1/2/0)
((d ₃₂ ,B),d ₁₆)	4	0	18(1/0/0)	19(1/2/0)
((d ₃₂ ,B),l,d ₁₆)	4	0	18(1/0/0)	19(1/2/0)
((d ₃₂ ,B),d ₃₂)	4	0	18(1/0/0)	19(1/2/0)
((d ₃₂ ,B),l,d ₃₂)	4	0	18(1/0/0)	19(1/2/0)

B = Base address; 0, An, PC, Xn, An + Xn, PC + Xn. Form does not affect timing.

l = Index; 0, Xn

% = Total head for effective address timing includes the head time for the operation.

NOTE: Xn cannot be in B and l at the same time. Scaling and size of Xn do not affect timing.

11.6.6 MOVE Instruction

The MOVE instruction timing table indicates the number of clock periods needed for the processor to calculate the destination effective address and perform the MOVE or MOVEA instruction, including the first level of indirection on memory indirect addressing modes. The fetch effective address table is needed on most MOVE operations (source, destination dependent). The destination effective addresses are divided by their formats (refer to **2.5 Effective Address Encoding Summary**). For instruction-cache case and for no-cache case, the total number of clock cycles is outside the parentheses. The number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). The read, prefetch, and write cycles are included in the total clock cycle number.

All timing data assumes two-clock reads and writes.

MOVE Source, Destination	Head	Tail	I-Cache Case	No-Cache Case
--------------------------	------	------	--------------	---------------

SINGLE EFFECTIVE ADDRESS INSTRUCTION FORMAT

MOVE Rn, Dn	2	0	2(0/0/0)	2(0/1/0)
MOVE Rn, An	2	0	2(0/0/0)	2(0/1/0)
* MOVE EA, An	0	0	2(0/0/0)	2(0/1/0)
* MOVE EA, Dn	0	0	2(0/0/0)	2(0/1/0)
MOVE Rn, (An)	0	1	3(0/0/1)	4(0/1/1)
* MOVE SOURCE, (An)	2	0	4(0/0/1)	5(0/1/1)
MOVE Rn, (An) +	0	1	3(0/0/1)	4(0/1/1)
* MOVE SOURCE, (An) +	2	0	4(0/0/1)	5(0/1/1)
MOVE Rn, -(An)	0	2	4(0/0/1)	4(0/1/1)
* MOVE SOURCE, -(An)	2	0	4(0/0/1)	5(0/1/1)
* MOVE EA, (d16, An)	2	0	4(0/0/1)	5(0/1/1)
* MOVE EA, XXX.W	2	0	4(0/0/1)	5(0/1/1)
* MOVE EA, XXX.L	0	0	6(0/0/1)	7(0/2/1)

BRIEF FORMAT EXTENSION WORD

* MOVE EA, (dg, An, Xn)	4	0	6(0/0/1)	7(0/1/1)
-------------------------	---	---	----------	----------

11.0.0 MOVE Instruction (Continued)

MOVE Source, Destination	Head	Tail	I-Cache Case	No-Cache Case
--------------------------	------	------	--------------	---------------

FULL FORMAT EXTENSION WORD(S)

* MOVE EA, (d16,An) or (d16,PC)	2	0	8(0/0/1)	9(0/2/1)
* MOVE EA, (d16,An,Xn) or (d16,PC,Xn)	2	0	8(0/0/1)	9(0/2/1)
* MOVE EA, ([d16,An],Xn) or ([d16,PC],Xn)	2	0	10(1/0/1)	11(1/2/1)
* MOVE EA, ([d16,An],d16) or ([d16,PC],d16)	2	0	12(1/0/1)	14(1/2/1)
* MOVE EA, ([d16,An],Xn,d16) or ([d16,PC],Xn,d16)	2	0	12(1/0/1)	14(1/2/1)
* MOVE EA, ([d16,An],d32) or ([d16,PC],d32)	2	0	14(1/0/1)	16(1/3/1)
* MOVE EA, ([d16,An],Xn,d32) or ([d16,PC],Xn,d32)	2	0	14(1/0/1)	16(1/3/1)
* MOVE EA, (B)	4	0	8(0/0/1)	9(0/1/1)
* MOVE EA, (d16,B)	4	0	10(0/0/1)	12(0/2/1)
* MOVE EA, (d32,B)	4	0	14(0/0/1)	16(0/2/1)
* MOVE EA, ([B])	4	0	10(1/0/1)	11(1/1/1)
* MOVE EA, ([B],l)	4	0	10(1/0/1)	11(1/1/1)
* MOVE EA, ([B],d16)	4	0	12(1/0/1)	14(1/2/1)
* MOVE EA, ([B],l,d16)	4	0	12(1/0/1)	14(1/2/1)
* MOVE EA, ([B],d32)	4	0	14(1/0/1)	16(1/2/1)
* MOVE EA, ([B],l,d32)	4	0	14(1/0/1)	16(1/2/1)
* MOVE EA, ([d16,B])	4	0	12(1/0/1)	14(1/2/1)
* MOVE EA, ([d16,B],l)	4	0	12(1/0/1)	14(1/2/1)
* MOVE EA, ([d16,B],d16)	4	0	14(1/0/1)	17(1/2/1)
* MOVE EA, ([d16,B],l,d16)	4	0	14(1/0/1)	17(1/2/1)
* MOVE EA, ([d16,B],d32)	4	0	16(1/0/1)	19(1/3/1)
* MOVE EA, ([d16,B],l,d32)	4	0	16(1/0/1)	19(1/3/1)
* MOVE EA, ([d32,B])	4	0	16(1/0/1)	18(1/2/1)
* MOVE EA, ([d32,B],l)	4	0	16(1/0/1)	18(1/2/1)
* MOVE EA, ([d32,B],d16)	4	0	18(1/0/1)	21(1/3/1)
* MOVE EA, ([d32,B],l,d16)	4	0	18(1/0/1)	21(1/3/1)
* MOVE EA, ([d32,B],d32)	4	0	20(1/0/1)	23(1/3/1)
* MOVE EA, ([d32,B],l,d32)	4	0	20(1/0/1)	23(1/3/1)

* Add Fetch Effective Address Time
Rn Is a Data or Address Register

SOURCE Is Memory or Immediate Data Address Mode
EA Is any Effective Address

11.6.7 Special-Purpose MOVE Instruction

The special-purpose MOVE timing table indicates the number of clock periods needed for the processor to fetch, calculate, and perform the special-purpose MOVE operation on the control registers or specified effective address. Footnotes indicate when to account for the appropriate effective address times. The total number of clock cycles is outside the parentheses. The number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). The read, prefetch, and write cycles are included in the total clock cycle number.

All timing data assumes two-clock reads and writes.

Instruction	Head	Tail	I-Cache Case	No-Cache Case
EXG Ry,Rx	4	0	4(0/0/0)	4(0/1/0)
MOVEC Cr,Rn	6	0	6(0/0/0)	6(0/1/0)
MOVEC Rn,Cr – A	6	0	6(0/0/0)	6(0/1/0)
MOVEC Rn,Cr – B	4	0	12(0/0/0)	12(0/1/0)
MOVE CCR,Dn	2	0	4(0/0/0)	4(0/1/0)
* MOVE CCR,Mem	2	0	4(0/0/1)	5(0/1/1)
MOVE Dn,CCR	4	0	4(0/0/0)	4(0/1/0)
* MOVE EA,CCR	0	0	4(0/0/0)	4(0/1/0)
MOVE SR,Dn	2	0	4(0/0/0)	4(0/1/0)
* MOVE SR,Mem	2	0	4(0/0/1)	5(0/1/1)
# MOVE EA,SR	0	0	8(0/0/0)	10(0/2/0)
% + MOVEM EA,RL	2	0	8 + 4n(n/0/0)	8 + 4n(n/1/0)
% + MOVEM RL,EA	2	0	4 + 2n(0/0/n)	4 + 2n(0/1/n)
MOVEP.W Dn,(d16,An)	4	0	10(0/0/2)	10(0/1/2)
MOVEP.W (d16,An),Dn	2	0	10(2/0/0)	10(2/1/0)
MOVEP.L Dn,(d16,An)	4	0	14(0/0/4)	14(0/1/4)
MOVEP.L (d16,An),Dn	2	0	14(4/0/0)	14(4/1/0)
% MOVES EA,Rn	3	0	7(1/0/0)	7(1/1/0)
% MOVES Rn,EA	2	1	5(0/0/1)	6(0/1/1)
MOVE USP,An	4	0	4(0/0/0)	4(0/1/0)
MOVE An,USP	4	0	4(0/0/0)	4(0/1/0)
SWAP Dn	4	0	4(0/0/0)	4(0/1/0)

CR – A Control Registers USP, VBR, CAAR, MSP, and ISP

CR – B Control Registers SFC, DFC, and CACR

n Number of Register to Transfer (n>0)

RL Register List

* Add Calculate Effective Address Time

Add Fetch Effective Address Time

% Add Calculate Immediate Address Time

+ MOVEM EA,RL — For n Registers (n > 0) and w Wait States

I-Cache Case Timing = $w \leq 2: (8 + 4n)$

$w > 2: (8 + 4n) + (w - 2)n$

Tail = 0 for all Wait States

MOVEM RL,EA — For n Registers (n > 0) and w Wait States

I-Cache Case Timing = $w \leq 2: (4 + 2n) + (n - 1)w$

$w > 2: (4 + 2n) + (n - 1)w + (w - 2)$

Tail = $w \leq 2: (n - 1)w$

$w > 2: (n)w + (n)(w - 2)$

11.6.8 Arithmetical/Logical Instructions

The arithmetical/logical operation timing table indicates the number of clock periods needed for the processor to perform the specified arithmetical/logical instruction using the specified addressing mode. Footnotes indicate when to account for the appropriate fetch effective address or fetch immediate effective address times. For instruction-cache case and for no-cache case, the total number of clock cycles is outside the parentheses. The number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). The read, prefetch, and write cycles are included in the total clock cycle number.

All timing data assumes two-clock reads and writes.

Instruction		Head	Tail	I-Cache Case	No-Cache Case
ADD	Rn,Dn	2	0	2(0/0/0)	2(0/1/0)
ADDA.W	Rn,An	4	0	4(0/0/0)	4(0/1/0)
ADDA.L	Rn,An	2	0	2(0/0/0)	2(0/1/0)
* ADD	EA,Dn	0	0	2(0/0/0)	2(0/1/0)
* ADD.W	EA,An	0	0	4(0/0/0)	4(0/1/0)
* ADDA.L	EA,An	0	0	2(0/0/0)	2(0/1/0)
* ADD	Dn,EA	0	1	3(0/0/1)	4(0/1/1)
AND	Dn,Dn	2	0	2(0/0/0)	2(0/1/0)
* AND	EA,Dn	0	0	2(0/0/0)	2(0/1/0)
* AND	Dn,EA	0	1	3(0/0/1)	4(0/1/1)
EOR	Dn,Dn	2	0	2(0/0/0)	2(0/1/0)
* EOR	Dn,EA	0	1	3(0/0/1)	4(0/1/1)
OR	Dn,Dn	2	0	2(0/0/0)	2(0/1/0)
* OR	EA,Dn	0	0	2(0/0/0)	2(0/1/0)
* OR	Dn,EA	0	1	3(0/0/1)	4(0/1/1)
SUB	Rn,Dn	2	0	2(0/0/0)	2(0/1/0)
* SUB	EA,Dn	0	0	2(0/0/0)	2(0/1/0)

11.6.8 Arithmetical/Logical Instructions (Continued)

Instruction			Head	Tail	I-Cache Case	No-Cache Case
*	SUB	Dn,EA	0	1	3(0/0/1)	4(0/1/1)
	SUBA.W	Rn,An	4	0	4(0/0/0)	4(0/1/0)
	SUBA.L	Rn,An	2	0	2(0/0/0)	2(0/1/0)
*	SUBA.W	EA,An	0	0	4(0/0/0)	4(0/1/0)
*	SUBA.L	EA,An	0	0	2(0/0/0)	2(0/1/0)
	CMP	Rn,Dn	2	0	2(0/0/0)	2(0/1/0)
*	CMP	EA,Dn	0	0	2(0/0/0)	2(0/1/0)
	CMPA	Rn,An	4	0	4(0/0/0)	4(0/1/0)
*	CMPA	EA,An	0	0	4(0/0/0)	4(0/1/0)
** +	CMP2	EA,Rn	2	0	20(1/0/0)	20(1/1/0)
* +	MULS.W	EA,Dn	2	0	28(0/0/0)	28(0/1/0)
** +	MULS.L	EA,Dn	2	0	44(0/0/0)	44(0/1/0)
* +	MULU.W	EA,Dn	2	0	28(0/0/0)	28(0/1/0)
** +	MULU.L	EA,Dn	2	0	44(0/0/0)	44(0/1/0)
+	DIVS.W	Dn,Dn	2	0	56(0/0/0)	56(0/1/0)
* +	DIVS.W	EA,Dn	0	0	56(0/0/0)	56(0/1/0)
** +	DIVS.L	Dn,Dn	6	0	90(0/0/0)	90(0/1/0)
** +	DIVS.L	EA,Dn	0	0	90(0/0/0)	90(0/1/0)
+	DIVU.W	Dn,Dn	2	0	44(0/0/0)	44(0/1/0)
* +	DIVU.W	EA,Dn	0	0	44(0/0/0)	44(0/1/0)
** +	DIVU.L	Dn,Dn	6	0	78(0/0/0)	78(0/1/0)
** +	DIVU.L	EA,Dn	0	0	78(0/0/0)	78(0/1/0)

*Add Fetch Effective Address Time

**Add Fetch Immediate Effective Address Time

+ Indicates Maximum Time (Actual time is data dependent)

11.6.9 Immediate Arithmetical/Logical Instructions

The immediate arithmetical/logical operation timing table indicates the number of clock periods needed for the processor to fetch the source immediate data value and to perform the specified arithmetic/logical operation using the specified destination addressing mode. Footnotes indicate when to account for the appropriate fetch effective or fetch immediate effective address times. For instruction-cache case and for no-cache case, the total number of clock cycles is outside the parentheses. The number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). The read, prefetch, and write cycles are included in the total clock cycle number.

All timing data assumes two-clock reads and writes.

Instruction	Head	Tail	I-Cache Case	No-Cache Case
MOVEQ #(data),Dn	2	0	2(0/0/0)	2(0/1/0)
ADDQ #(data),Rn	2	0	2(0/0/0)	2(0/1/0)
* ADDQ #(data),Mem	0	1	3(0/0/1)	4(0/1/1)
SUBQ #(data),Rn	2	0	2(0/0/0)	2(0/1/0)
* SUBQ #(data),Mem	0	1	3(0/0/1)	4(0/1/1)
** ADDI #(data),Dn	2	0	2(0/0/0)	2(0/1/0)
** ADDI #(data),Mem	0	1	3(0/0/1)	4(0/1/1)
** ANDI #(data),Dn	2	0	2(0/0/0)	2(0/1/0)
** ANDI #(data),Mem	0	1	3(0/0/1)	4(0/1/1)
** EORI #(data),Dn	2	0	2(0/0/0)	2(0/1/0)
** EORI #(data),Mem	0	1	3(0/0/1)	4(0/1/1)
** ORI #(data),Dn	2	0	2(0/0/0)	2(0/1/0)
** ORI #(data),Mem	0	1	3(0/0/1)	4(0/1/1)
** SUBI #(data),Dn	2	0	2(0/0/0)	2(0/1/0)
** SUBI #(data),Mem	0	1	3(0/0/1)	4(0/1/1)
** CMPI #(data),Dn	2	0	2(0/0/0)	2(0/1/0)
** CMPI #(data),Mem	0	0	2(0/0/0)	2(0/1/0)

*Add Fetch Effective Address Time

**Add Fetch Immediate Effective Address Time

11.6.10 Binary-Coded Decimal and Extended Instructions

The binary-coded decimal and extended instruction table indicates the number of clock periods needed for the processor to perform the specified operation using the given addressing modes. No additional tables are needed to calculate total effective execution time for these instructions. For instruction-cache case and for no-cache case, the total number of clock cycles is outside the parentheses. The number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). The read, prefetch, and write cycles are included in the total clock cycle number.

All timing data assumes two-clock reads and writes.

Instruction	Head	Tail	I-Cache Case	No-Cache Case
ABCD Dn,Dn	0	0	4(0/0/0)	4(0/1/0)
ABCD – (An), – (An)	2	1	13(2/0/1)	14(2/1/1)
SBCD Dn,Dn	0	0	4(0/0/0)	4(0/1/0)
SBCD – (An), – (An)	2	1	13(2/0/1)	14(2/1/1)
ADDX Dn,Dn	2	0	2(0/0/0)	2(0/1/0)
ADDX – (An), – (An)	2	1	9(2/0/1)	10(2/1/1)
SUBX Dn,Dn	2	0	2(0/0/0)	2(0/1/0)
SUBX – (An), – (An)	2	1	9(2/0/1)	10(2/1/1)
CMPM (An) + , (An) +	0	0	8(2/0/0)	8(2/1/0)
PACK Dn,Dn,#(data)	6	0	6(0/0/0)	6(0/1/0)
PACK – (An), – (An),#(data)	2	1	11(1/0/1)	11(1/1/1)
UNPK Dn,Dn,#(data)	8	0	8(0/0/0)	8(0/1/0)
UNPK – (An), – (An),#(data)	2	1	11(1/0/1)	11(1/1/1)

11.6.11 Single Operand Instructions

The single operand instruction table indicates the number of clock periods needed for the processor to perform the specified operation on the given addressing mode. Footnotes indicate when it is necessary to account for the appropriate effective address time. For instruction-cache case and for no-cache case, the total number of clock cycles is outside the parentheses. The number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). The read, prefetch, and write cycles are included in the total clock cycle number.

All timing data assumes two-clock reads and writes.

Instruction			Head	Tail	I-Cache Case	No-Cache Case
	CLR	Dn	2	0	2(0/0/0)	2(0/1/0)
**	CLR	Mem	0	1	3(0/0/1)	4(0/1/1)
	NEG	Dn	2	0	2(0/0/0)	2(0/1/0)
*	NEG	Mem	0	1	3(0/0/1)	4(0/1/1)
	NEGX	Dn	2	0	2(0/0/0)	2(0/1/0)
*	NEGX	Mem	0	1	3(0/0/1)	4(0/1/1)
	NOT	Dn	2	0	2(0/0/0)	2(0/1/0)
*	NOT	Mem	0	1	3(0/0/1)	4(0/1/1)
	EXT	Dn	4	0	4(0/0/0)	4(0/1/0)
	NBCD	Dn	0	0	6(0/0/0)	6(0/1/0)
	Scc	Dn	4	0	4(0/0/0)	4(0/1/0)
**	Scc	Mem	0	1	5(0/0/1)	5(0/1/1)
	TAS	Dn	4	0	4(0/0/0)	4(0/1/0)
**	TAS	Mem	3	0	12(1/0/1)	12(1/1/1)
	TST	Dn	0	0	2(0/0/0)	2(0/1/0)
*	TST	Mem	0	0	2(0/0/0)	2(0/1/0)

*Add Fetch Effective Address Time

**Add Calculate Effective Address Time

11.6.12 Shift/Rotate Instructions

The shift/rotate instruction table indicates the number of clock periods needed for the processor to perform the specified operation on the given addressing mode. Footnotes indicate when it is necessary to account for the appropriate effective address time. The number of bits shifted does not affect the execution time, unless noted. For instruction-cache case and for no-cache case, the total number of clock cycles is outside the parentheses. The number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). The read, prefetch, and write cycles are included in the total clock cycle number.

All timing data assumes two-clock reads and writes.

	Instruction	Head	Tail	I-Cache Case	No-Cache Case
	LSd #(data),Dy	4	0	4(0/0/0)	4(0/1/0)
%	LSd Dx,Dy	6	0	6(0/0/0)	6(0/1/0)
+	LSd Dx,Dy	8	0	8(0/0/0)	8(0/1/0)
*	LSd Mem by 1	0	0	4(0/0/1)	4(0/1/1)
	ASL #(data),Dy	2	0	6(0/0/0)	6(0/1/0)
	ASL Dx,Dy	4	0	8(0/0/0)	8(0/1/0)
*	ASL Mem by 1	0	0	6(0/0/1)	6(0/1/1)
	ASR #(data),Dy	4	0	4(0/0/0)	4(0/1/0)
%	ASR Dx,Dy	6	0	6(0/0/0)	6(0/1/0)
+	ASR Dx,Dy	10	0	10(0/0/0)	10(0/1/0)
*	ASR Mem by 1	0	0	4(0/0/1)	4(0/1/1)
	ROd #(data),Dy	4	0	6(0/0/0)	6(0/1/0)
	ROd Dx,Dy	6	0	8(0/0/0)	8(0/1/0)
*	ROd Mem by 1	0	0	6(0/0/1)	6(0/1/1)
	ROXd Dn	10	0	12(0/0/0)	12(0/1/0)
*	ROXd Mem by 1	0	0	4(0/0/0)	4(0/1/0)

- d Direction of shift/rotate: L or R
- * Add Fetch Effective Address Time
- % Indicates shift count is less than or equal to the size of data
- +

11.6.13 Bit Manipulation Instructions

The bit manipulation instruction table indicates the number of clock periods needed for the processor to perform the specified bit operation on the given addressing mode. Footnotes indicate when it is necessary to account for the appropriate effective address time. For instruction-cache case and for no-cache case, the total number of clock cycles is outside the parentheses. The number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). The read, prefetch, and write cycles are included in the total clock cycle number.

All timing data assumes two-clock reads and writes.

	Instruction	Head	Tail	I-Cache Case	No-Cache Case
	BTST #(data),Dn	4	0	4(0/0/0)	4(0/1/0)
	BTST Dn,Dn	4	0	4(0/0/0)	4(0/1/0)
#	BTST #(data),Mem	0	0	4(0/0/0)	4(0/1/0)
*	BTST Dn,Mem	0	0	4(0/0/0)	4(0/1/0)
	BCHG #(data),Dn	6	0	6(0/0/0)	6(0/1/0)
	BCHG Dn,Dn	6	0	6(0/0/0)	6(0/1/0)
#	BCHG #(data),Mem	0	0	6(0/0/1)	6(0/1/1)
*	BCHG Dn,Mem	0	0	6(0/0/1)	6(0/1/1)
	BCLR #(data),Dn	6	0	6(0/0/0)	6(0/1/0)
	BCLR Dn,Dn	6	0	6(0/0/0)	6(0/1/0)
#	BCLR #(data),Mem	0	0	6(0/0/1)	6(0/1/1)
*	BCLR Dn,Mem	0	0	6(0/0/1)	6(0/1/1)
	BSET #(data),Dn	6	0	6(0/0/0)	6(0/1/0)
	BSET Dn,Dn	6	0	6(0/0/0)	6(0/1/0)
#	BSET #(data),Mem	0	0	6(0/0/1)	6(0/1/1)
*	BSET Dn,Mem	0	0	6(0/0/1)	6(0/1/1)

*Add Fetch Effective Address Time

#Add Fetch Immediate Effective Address Time

11.6.14 Bit Field Manipulation Instructions

The bit field manipulation instruction table indicates the number of clock periods needed for the processor to perform the specified bit field operation using the given addressing mode. Footnotes indicate when it is necessary to account for the appropriate effective address time. For instruction-cache case and for no-cache case, the total number of clock cycles is outside the parentheses. The number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). The read, prefetch, and write cycles are included in the total clock cycle number.

All timing data assumes two-clock reads and writes.

Instruction		Head	Tail	I-Cache Case	No-Cache Case
BFTST	Dn	8	0	8(0/0/0)	8(0/1/0)
* BFTST	Mem (<5 Bytes)	6	0	10(1/0/0)	10(1/1/0)
* BFTST	Mem (5 Bytes)	6	0	14(2/0/0)	14(2/1/0)
BFCHG	Dn	14	0	14(0/0/0)	14(0/1/0)
* BFCHG	Mem (<5 Bytes)	6	0	14(1/0/1)	14(1/1/1)
* BFCHG	Mem (5 Bytes)	6	0	22(2/0/2)	22(2/1/2)
BFCLR	Dn	14	0	14(0/0/0)	14(0/1/0)
* BFCLR	Mem (<5 Bytes)	6	0	14(1/0/1)	14(1/1/1)
* BFCLR	Mem (5 Bytes)	6	0	22(2/0/2)	22(2/1/2)
BFSET	Dn	14	0	14(0/0/0)	14(0/1/0)
* BFSET	Mem (<5 Bytes)	6	0	14(1/0/1)	14(1/1/1)
* BFSET	Mem (5 Bytes)	6	0	22(2/0/2)	22(2/1/2)
BFEXTS	Dn	10	0	10(0/0/0)	10(0/1/0)
* BFEXTS	Mem (<5 Bytes)	6	0	12(1/0/0)	12(1/1/0)
* BFEXTS	Mem (5 Bytes)	6	0	18(2/0/0)	18(2/1/0)
BFEXTU	Dn	10	0	10(0/0/0)	10(0/1/0)
* BFEXTU	Mem (<5 Bytes)	6	0	12(1/0/0)	12(1/1/0)
* BFEXTU	Mem (5 Bytes)	6	0	18(2/0/0)	18(2/1/0)
BFINS	Dn	12	0	12(0/0/0)	12(0/1/0)
* BFINS	Mem (<5 Bytes)	6	0	12(1/0/1)	12(1/1/1)
* BFINS	Mem (5 Bytes)	6	0	18(2/0/2)	18(2/1/2)
BFFFO	Dn	20	0	20(0/0/0)	20(0/1/0)
* BFFFO	Mem (<5 Bytes)	6	0	22(1/0/0)	22(1/1/0)
* BFFFO	Mem (5 Bytes)	6	0	28(2/0/0)	28(2/1/0)

*Add Calculate Immediate Effective Address Time

NOTE: A bit field of 32 bits may span 5 bytes that require two operand cycles to access or may span 4 bytes that require only one operand cycle to access.

11.6.15 Conditional Branch Instructions

The conditional branch instruction table indicates the number of clock periods needed for the processor to perform the specified branch on the given branch size, with complete execution times given. No additional tables are needed to calculate total effective execution time for these instructions. For instruction-cache case and for no-cache case, the total number of clock cycles is outside the parentheses. The number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). The read, prefetch, and write cycles are included in the total clock cycle number.

All timing data assumes two-clock reads and writes.

Instruction	Head	Tail	I-Cache Case	No-Cache Case
Bcc (Taken)	6	0	6(0/0/0)	8(0/2/0)
Bcc.B (Not Taken)	4	0	4(0/0/0)	4(0/1/0)
Bcc.W (Not Taken)	6	0	6(0/0/0)	6(0/1/0)
Bcc.L (Not Taken)	6	0	6(0/0/0)	8(0/2/0)
DBcc (cc = False, Count Not Expired)	6	0	6(0/0/0)	8(0/2/0)
DBcc (cc = False, Count Expired)	10	0	10(0/0/0)	13(0/3/0)
DBcc (cc = True)	6	0	6(0/0/0)	8(0/1/0)

11.6.16 Control Instructions

The control instruction table indicates the number of clock periods needed for the processor to perform the specified operation. Footnotes indicate when it is necessary to account for the appropriate effective address time. For instruction-cache case and for no-cache case, the total number of clock cycles is outside the parentheses. The number of read, prefetch, and write cycles is given inside the parentheses as (r p w). The read, prefetch, and write cycles are included in the total clock cycle number.

All timing data assumes two-clock reads and writes.

Instruction	Head	Tail	I-Cache Case	No-Cache Case
ANDI to SR	4	0	12(0/0/0)	14(0/2/0)
EORI to SR	4	0	12(0/0/0)	14(0/2/0)
ORI to SR	4	0	12(0/0/0)	14(0/2/0)
ANDI to CCR	4	0	12(0/0/0)	14(0/2/0)
EORI to CCR	4	0	12(0/0/0)	14(0/2/0)
ORI to CCR	4	0	12(0/0/0)	14(0/2/0)
BSR	2	0	6(0/0/1)	9(0/2/1)
## CAS (Successful Compare)	1	0	13(1/0/1)	13(1/1/1)
## CAS (Unsuccessful Compare)	1	0	11(1/0/0)	11(1/1/0)
+ CAS2 (Successful Compare)	2	0	24(2/0/2)	26(2/2/2)
+ CAS2 (Unsuccessful Compare)	2	0	24(2/0/0)	24(2/2/0)
CHK Dn,Dn (No Exception)	8	0	8(0/0/0)	8(0/1/0)
+ CHK Dn,Dn (Exception Taken)	4	0	28(1/0/4)	30(1/3/4)
* CHK EA,Dn (No Exception)	0	0	8(0/0/0)	8(0/1/0)
* + CHK EA,Dn (Exception Taken)	0	0	28(1/0/4)	30(1/3/4)
# + CHK2 Mem,Rn (No Exception)	2	0	18(1/0/0)	18(1/1/0)
# + CHK2 Mem,Rn (Exception Taken)	2	0	40(2/0/4)	42(2/3/4)
% JMP	4	0	4(0/0/0)	6(0 2 0)
% JSR	0	0	4(0/0/1)	7(0 2 1)
** LEA	2	0	2(0/0/0)	2(0/1 0)
LINK.W	0	0	4(0/0/1)	5(0 1 1)
LINK.L	2	0	6(0/0/1)	7(0 2 1)
NOP	0	0	2(0/0/0)	2(0 1 0)
** PEA	0	2	4(0/0/1)	4(0 1 1)
RTD	2	0	10(1/0/0)	12(1 2 0)
RTR	1	0	12(2/0/0)	14(2 2 0)
RTS	1	0	9(1/0/0)	11(1 2 0)
UNLK	0	0	5(1/0/0)	5(1 1 0)

- + Indicates Maximum Time
- * Add Fetch Effective Address Time
- ** Add Calculate Effective Address Time
- # Add Fetch Immediate Address Time
- ## Add Calculate Immediate Address Time
- % Add Jump Effective Address Time

11.6.1/ Exception-Related Instructions and Operations

The exception-related instruction and operation table indicates the number of clock periods needed for the processor to perform the specified exception-related action. No additional tables are needed to calculate total effective execution time for these operations. For instruction-cache case and for no-cache case, the total number of clock cycles is outside the parentheses. The number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). The read, prefetch, and write cycles are included in the total clock cycle number.

All timing data assumes two-clock reads and writes.

Instruction/Operation	Head	Tail	I-Cache Case	No-Cache Case
BKPT	1	0	9(1/0/0)	9(1/0/0)
Interrupt (I-Stack)	0	0	23(2/0/4)	24(2/2/4)
Interrupt (M-Stack)	0	0	33(2/0/8)	34(2/2/8)
RESET Instruction	0	0	518(0/0/0)	518(0/1/0)
STOP	0	0	8(0/0/0)	8(0/2/0)
TRACE	0	0	22(1/0/5)	24(1/2/5)
TRAP #n	0	0	18(1/0/4)	20(1/2/4)
Illegal Instruction	0	0	18(1/0/4)	20(1/2/4)
A-Line Trap	0	0	18(1/0/4)	20(1/2/4)
F-Line Trap	0	0	18(1/0/4)	20(1/2/4)
Privilege Violation	0	0	18(1/0/4)	20(1/2/4)
TRAPcc (Trap)	2	0	22(1/0/5)	24(1/2/5)
TRAPcc (No Trap)	4	0	4(0/0/0)	4(0/1/0)
TRAPcc.W (Trap)	5	0	24(1/0/5)	26(1/3/5)
TRAPcc.W (No Trap)	6	0	6(0/0/0)	6(0/1/0)
TRAPcc.L (Trap)	6	0	26(1/0/5)	28(1/3/5)
TRAPcc.L (No Trap)	8	0	8(0/0/0)	8(0/2/0)
TRAPV (Trap)	2	0	22(1/0/5)	24(1/2/5)
TRAPV (No Trap)	4	0	4(0/0/0)	4(0/1/0)

11.6.18 Save and Restore Operations

The save and restore operation table indicates the number of clock periods needed for the processor to perform the specified state save or to return from exception, with complete execution times and stack length given. No additional tables are needed to calculate total effective execution time for these operations. For instruction-cache case and for no-cache case, the total number of clock cycles is outside the parentheses. The number of read, prefetch, and write cycles is given inside the parentheses as (r/p/w). The read, prefetch, and write cycles are included in the total clock cycle number.

All timing data assumes two-clock reads and writes.

Operation	Head	Tail	I-Cache Case	No-Cache Case
Bus Cycle Fault (Short)	0	0	36(1/0/10)	38(1/2/10)
Bus Cycle Fault (Long)	0	0	62(1/0/24)	64(1/2/24)
RTE (Normal Four Word)	1	0	18(4/0/0)	20(4/2/0)
RTE (Six Word)	1	0	18(4/0/0)	20(4/2/0)
RTE (Throwaway)	1	0	12(4/0/0)	12(4/0/0)
RTE (Coprocessor)	1	0	26(7/0/0)	26(7/2/0)
RTE (Short Fault)	1	0	36(10/0/0)	26(10/2/0)
RTE (Long Fault)	1	0	76(25/0/0)	76(25/2/0)

11.7 Address Translation Tree Search Timing

The time required for a search of the address translation tree depends on the configuration of the tree structure and the descriptors in the tree, the states of the used (U) and modified (M) bits in the descriptors, bus cycle time, and other factors. The large number of variables involved implies that search time can best be calculated by a program. To determine the time required for the MC68030 to perform the table search for a specific configuration, the following interactive program can be used. It is a shell script suitable for use with **sh(1)** on either UNIX[®] System V or BSD 4.2. To use the program, run the script and answer the questions about the system configuration and current state. The values shown in square brackets at the ends of the question lines are the default values that the program uses when carriage returns are entered.

The shell script assumes that the data bus between the MC68030 and memory is 32 bits wide. To calculate the search time for a narrower bus, enter the appropriate multiple of the bus cycle time in response to the bus cycle time

[®]UNIX is a registered trademark of AT&T Bell Laboratories.

prompt. Use the time required for two bus cycles in the case of a 16-bit data bus. Use the time required for four bus cycles in the case of an 8-bit data bus.

The times provided by this program include all phases of the translation tree search. With various mask versions of the MC68030, times may differ slightly from those calculated by the program.

```
#####
#
# This Shell script is suitable for use with sh(1) on either System V or
# BSD 4.2. When run, it will prompt for several parameters, print a
# configuration message, and then print the number of clocks and bus
# cycles required for the table search. Questions may be answered with
# a carriage return, and the default in square brackets will be selected.
#
# The following things should be noted by the user:
#
# 1. This script gives an approximation for the time taken for a table
# search and associated overhead for a miss in the ATC. The exact time
# will vary with the instruction sequence being executed at the time of the
# miss, and may vary plus or minus 2 clocks (see pre-walk overhead, below).
#
# 2. It will give accurate times for normal table walks (due to misses
# in the ATC) and for PLOAD table walks but not for PTEST table walks.
# Table walks due to the PTEST instruction will be somewhat longer.
#
# 3. It does little error checking. It is possible to describe
# inconsistent and impossible configurations in the script.
#
#
echo -n "Enter bus cycle time (in clocks) [2]: "
read bus
if test ! "$bus"; then
    bus=2
fi

echo -n "Enter 1 if there is a function code lookup, 0 otherwise [0]: "
read fcl
if test ! "$fcl"; then
    fcl=0
fi

echo -n "Enter number of long descriptors (page and pointer), including FCL ones [1]: "
read long
if test ! "$long"; then
    long=1
fi

echo -n "Enter number of short descriptors (page or pointer), including FCL ones [1]: "
read short
if test ! "$short"; then
    short=1
fi
```



```
echo -n "Enter 1 if there is a long indirect descriptor, 0 otherwise [0]: "
read l_ind
if test ! "$l_ind"; then
    l_ind=0
fi

echo -n "Enter 1 if there is a short indirect descriptor, 0 otherwise [0]: "
read s_ind
if test ! "$s_ind"; then
    s_ind=0
fi

echo -n "Enter number of cleared ubits encountered in pointer descriptors [0]: "
read pointer_ubits
if test ! "$pointer_ubits"; then
    pointer_ubits=0
fi

echo -n "Enter 1 if the page descriptor ubit and/or mbit is clear, 0 otherwise [0]: "
read page_m_ubit
if test ! "$page_m_ubit"; then
    page_m_ubit=0
fi

echo -n "Enter 1 if the page descriptor is encountered unexpectedly, 0 otherwise [0]: "
read et
if test ! "$et"; then
    et=0
fi

echo -n "Enter 1 if the page descriptor is long (and no rp et) [0]: "
read long_page
if test ! "$long_page"; then
    long_page=0
fi

#####
#
# Print Configuration message.
#

levels=`expr $short + $long + $l_ind + $s_ind`

if test $fcl -eq 1; then
    tmp1=" (one for FCL)"
else
    tmp1=""
fi

out1="Configuration: $levels levels $tmp1 - "

if test $long -ne 0 ; then
    out1="$out1 $long long descriptor(s) "
fi

if test $short -ne 0 ; then
    out1="$out1 $short short descriptor(s)"
fi

if test $l_ind -eq 1 ; then
    out1="$out1 long indirection"
elif test $s_ind -eq 1 ; then
    out1="$out1 short indirection"
fi
```



```
11 test $pointer_ubits -ne 0 ; then
    out2="$out2 $pointer_ubits pointer ubits clear, "
fi

if test $page_m_ubit -eq 1 ; then
    out2="$out2 page ubit and/or mbit clear, "
fi

if test $set -eq 1 ; then
    out2="$out2 early termination, "
fi

if test $long_page -eq 1 ; then
    out2="$out2 page is long;"
else
    out2="$out2 page is short;"
fi

out3="$bus clock bus cycle time."

echo
echo $out1
echo "          " $out2
echo "          " $out3

#####
#
# Calculate result.
#
# Variables:
#
#   cough --- the number of clocks from the start of the bus cycle that will miss to
#             the first clock of the first micro-instruction.
#
#   startup -- microcode startup overhead common to all flows
#
#   termination -- microcode termination overhead common to all flows
#
#   bus_max_4 bus_max_3 the maximum value of the bus cycle time (in clocks) and
#                       4 or 3, respectively.
#
#
bus_reads=0
bus_writes=0
ind_clocks=0

# time from BEGINNING of bus cycle which misses to first box
# this is 6 to 9 clocks depending on i- and d-state at miss-- use 7 as average
cough=7

# 4 boxes of startup, when no FCL.
startup=8

# 4 boxes of termination.
termination=8

# Bus accesses begin sooner if FCL - no limit check.
if test $fcl -eq 1 ; then
    startup='expr $startup - 2'
fi

# calculate max((bus-4),0) for overlap
bus_max_4='expr $bus - 4'

if test $bus_max_4 -lt 0; then
    bus_max_4=0
fi
```



```
# calculate max((bus-3),0) for overlap
bus_max_3='expr $bus - 3'

if test $bus_max_3 -lt 0; then
    bus_max_3=0
fi

overhead='expr $cough + $startup + $termination'

# number of clock due to long descriptors
l_clocks='expr $long \* \(( 6 + $bus + $bus_max_4 \) )'

#long page is one box less than long pointer
if test $long_page -eq 1; then
    l_clocks='expr $l_clocks - 2'
fi

bus_reads='expr $bus_reads + \(( $long \* 2 \) )'

# number of clock due to short descriptors
s_clocks='expr $short \* \(( 3 + $bus \) )'
bus_reads='expr $bus_reads + $short'

# total clocks due to descriptor fetches
t_clocks='expr $l_clocks + $s_clocks'

if test $t_clocks -eq 0 ; then
    if test $et -ne 1 ; then
        echo Error: 0 bus accesses must imply unexpected page encountered.
    fi
    et=0
fi

# now caculate clocks due to setting u bits in pointer descriptor
u_clocks='expr $pointer_ubits \* \(( 4 + $bus_max_3 \) )'
bus_writes='expr $bus_writes + $pointer_ubits + $page_m_ubit'

# clocks due to setting u/m bits in page descriptor
page_clocks='expr $page_m_ubit \* \(( 2 + $bus_max_3 \) )'
bus_writes='expr $bus_writes + $page_m_ubit'

# clocks due to indirect level (long)
if test $l_ind -ne 0; then
    ind_clocks='expr 2 + \(( $bus \* 2 \) )'
    bus_reads='expr $bus_reads + 2'
fi

# clocks due to indirect level (short)
if test $s_ind -ne 0; then
    ind_clocks='expr 3 + $bus'
    bus_reads='expr $bus_reads + 1'
fi

# early termination penalty
if test $et -eq 1; then
    et_delay=3
else
    et_delay=0
fi
```

```
#####
#
# Perform the calculation.
#

clocks='expr $overhead \
+ $l_clocks \
+ $s_clocks \
+ $u_clocks \
+ $page_clocks \
+ $ind_clocks \
+ $et_delay`

out="      Clocks required (from beginning of missed bus cycle): $clocks"
echo
echo $out

write_accesses='expr $pointer_ubits + $page_m_ubit`

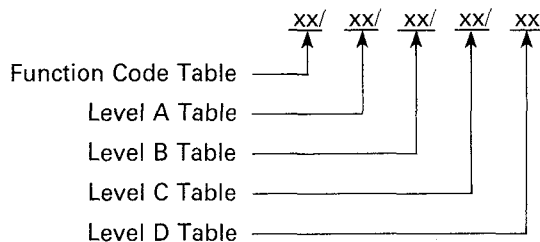
out="      Bus Reads:                                $bus_reads"
echo $out

print_total=0
if test $write_accesses -ne 0 ; then
    out="      Bus Writes:                                $write_accesses"
    echo $out
    print_total=1
fi

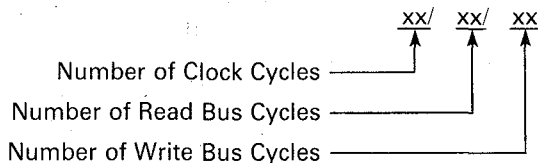
bus_accesses='expr $bus_reads + $write_accesses`

if test $print_total -eq 1 ; then
    out="      Total Bus Cycles:                                $bus_accesses"
    echo $out
fi
```

The following table gives some sample times obtained using the shell script. Each row of the table indicates a translation table configuration. The identifier on each row consists of five positions. Each position may have either an "X", meaning that there is no table at the level; an "S", meaning that the table at the level is composed of short-format descriptors; or an "L", meaning that the table at the level is composed of long-format descriptors. The format of the entries is:



Each entry in the table consists of three numbers that give the number of clock cycles, the number of bus reads, and the number of bus writes required for a table search. An RMC cycle to set the U bit is counted as one read and one write. The format of the entries is:



The table is calculated based on the following assumptions:

1. Bus cycle time is two clock cycles,
2. There are no indirect descriptors,
3. There are no page descriptors encountered unexpectedly (no early termination), and
4. The memory port is 32 bits wide.

Table Format	All U and M Bits Must be Set	Page U and M Bits Only Must be Set	No U and M Bits Must be Set
LLxxx	41/4/2	37/4/1	35/4/0
LLLxx	53/6/3	45/6/1	43/6/0
LLLLx	65/8/4	53/8/1	51/8/0
LLLLL	77/10/5	61/10/1	59/10/0
SSxxx	37/2/2	33/2/1	31/2/0
SSSxx	46/3/3	38/3/1	36/3/0
SSSSx	55/4/4	43/4/1	41/4/0
SSSSS	64/5/5	48/5/1	46/5/0
xSSxx	39/2/2	35/2/1	33/2/0
xSLxx	40/3/2	36/3/1	34/3/0
xLSxx	42/3/2	38/3/1	36/3/0
xLLxx	43/4/2	39/4/1	37/4/0
xSSSx	48/3/3	40/3/1	38/3/0
xSSLx	49/4/3	41/4/1	39/4/0
xSLSx	51/4/3	43/4/1	41/4/0
xSLLx	52/5/3	44/5/1	44/5/0
xLSSx	51/4/3	43/4/1	41/4/0
xLSLx	52/5/3	44/5/1	42/5/0
xLLSx	54/5/3	46/5/1	44/5/0
xLLLx	55/6/3	47/6/1	45/6/0

..... ..IMU Effective Address Calculation

The calculate effective address table for MMU instructions lists the number of clock periods needed for the processor to calculate various effective addresses. Fetch time is only included for the first level of indirection on memory indirect addressing modes. The total number of clock cycles is outside the parentheses. This total includes the number of read, prefetch, and write cycles, which are shown inside the parentheses as (r/pr/w).

Address Mode	Head	Tail	I-Cache Case	No-Cache Case
(An)	4 + op head	0	4(0/0/0)	4(0/1/0)
(d ₁₆ ,An)	4 + op head	0	4(0/0/0)	4(0/1/0)
(xxx).W	4 + op head	0	4(0/0/0)	4(0/1/0)
(xxx).L	6 + op head	0	6(0/0/0)	6(0/2/0)
(dg,An,Xn)	4 + op head	0	4(0/0/0)	4(0/1/0)

FULL FORMAT EXTENSION WORD(S)

(d ₁₆ ,An)	4	0	8(0/0/0)	8(0/2/0)
(d ₁₆ ,An,Xn)	4	0	8(0/0/0)	8(0/2/0)
([d ₁₆ ,An])	4	0	12(1/0/0)	12(1/2/0)
([d ₁₆ ,An],Xn)	4	0	12(1/0/0)	12(1/2/0)
([d ₁₆ ,An],d ₁₆)	2	0	12(1/0/0)	12(1/2/0)
([d ₁₆ ,An],Xn,d ₁₆)	4	0	12(1/0/0)	12(1/2/0)
([d ₁₆ ,An],d ₃₂)	4	0	14(1/0/0)	14(1/3/0)
([d ₁₆ ,An],Xn,d ₃₂)	4	0	14(1/0/0)	14(1/3/0)
(B)	8 + op head	0	8(0/0/0)	8(0/1/0)
(d ₁₆ ,B)	6	0	10(0/0/0)	10(0/2/0)
(d ₃₂ ,B)	6	0	16(0/0/0)	16(0/2/0)
([B])	6	0	12(1/0/0)	12(1/1/0)
([B],l)	6	0	12(1/0/0)	12(1/1/0)
([B],d ₁₆)	6	0	12(1/0/0)	12(1/2/0)
([B],l,d ₁₆)	6	0	12(1/0/0)	12(1/2/0)
([B],d ₃₂)	6	0	14(1/0/0)	14(1/2/0)
([B],l,d ₃₂)	6	0	14(1/0/0)	14(1/2/0)
([d ₁₆ ,B])	6	0	14(1/0/0)	14(1/2/0)
([d ₁₆ ,B],l)	6	0	14(1/0/0)	14(1/2/0)
([d ₁₆ ,B],d ₁₆)	6	0	14(1/0/0)	14(1/2/0)
([d ₁₆ ,B],l,d ₁₆)	6	0	14(1/0/0)	14(1/2/0)
([d ₁₆ ,B],d ₃₂)	6	0	16(1/0/0)	16(1/3/0)

11.7.1 MMU Effective Address Calculation (Continued)

Address Mode	Head	Tail	I-Cache Case	No-Cache Case
--------------	------	------	--------------	---------------

FULL FORMATION EXTENSION WORD(S) (CONTINUED)

{(d ₃₂ ,B)}	6	0	20(1/0/0)	20(1/2/0)
{(d ₃₂ ,B),I}	6	0	20(1/0/0)	20(1/2/0)
{(d ₃₂ ,B),d ₁₆ }	6	0	20(1/0/0)	20(1/3/0)
{(d ₃₂ ,B),I,d ₁₆ }	6	0	20(1/0/0)	20(1/3/0)
{(d ₃₂ ,B),d ₃₂ }	6	0	22(1/0/0)	22(1/3/0)
{(d ₃₂ ,B),I,d ₃₂ }	6	0	22(1/0/0)	22(1/3/0)

B = Base address; O, An, Xn, An + Xn. Form does not affect timing.

I = Index; O, Xn

*No separation on effective address and operation in timing. Head and tail are the operation's.

NOTE: Xn cannot be in B and I at the same time. Scaling and size of Xn do not affect timing.

11.7.2 MMU Instruction Timing

The MMU instruction timing table lists the numbers of clock periods needed for the MMU to perform the MMU instructions. The total number of clock cycles is outside the parentheses. It includes the numbers of read, prefetch, and write cycles, which are shown inside the parentheses as (r/pr/w).

Instruction	Head	Tail	I-Cache Case	No-Cache Case
PMOVE (from CRP, SRP)*	0	0	4(0/0/2)	5(0/1/2)
PMOVE (to CRP, SRP, valid)*	0	0	12(2/0/0)	14(2/2/0)
PMOVE (to CRP, SRP, invalid) ¹ *	0	0	28(3/0/4)	30(3/2/4)
PMOVE (from TT0, TT1)*	0	0	8(0/0/1)	8(0/1/1)
PMOVE (to TT0, TT1)*	0	0	12(1/0/0)	14(1/2/0)
PMOVE (from MMUSR)*	2	0	4(0/0/1)	5(0/1/1)
PMOVE (to MMUSR)*	0	0	6(1/0/0)	6(1/1/0)
PMOVE (from TC)*	2	0	4(0/0/1)	5(0/1/1)
PMOVE (to TC, valid) ² *	0	0	38(1/0/0)	40(1/2/0)
PMOVE (to TC, invalid) ³ *	0	0	56(2/0/4)	58(2/2/4)
PMOVE (to TC) ⁴ *	0	0	14(1/0/0)	16(1/2/0)
PFLUSHA	0	0	12(0/0/0)	14(0/2/0)
PFLUSH (fc),#(mask) (fc is immediate or data register)	0	0	16(0/0/0)	18(0/2/0)
PFLUSH (fc),#(mask) (fc is in SFC or DFC register)	0	0	20(0/0/0)	22(0/2/0)
PFLUSH (fc),#(mask),(ea) (fc is immediate or data register)*	0	0	16(0/0/0)	18(0/2/0)
PFLUSH (fc),#(mask),(ea) (fc is in SFC or DFC register)*	0	0	20(0/0/0)	22(0/2/0)
PLOAD[R:W] (fc),(ea) (fc is immediate or data register)**	0	0	8(0/0/0)	10(0/2/0)
PLOAD[R:W] (fc),(ea) (fc is in SFC or DFC register)**	0	0	12(0/0/0)	14(0/2/0)
PTEST[R:W] (fc),(ea),#6 * ***	0	0	88(12/0/0)	88(12/1/0)
PTEST[R:W] (fc),(ea),#0*	0	0	22(0/0/0)	22(0/1/0)

NOTES:

1. Attempt to load invalid root pointer.
2. Translation enabled.
3. Number is maximum, assuming valid page size but Tlx fields do not add up to 32. Translation enabled.
4. Translation disabled.

*Add the appropriate effective address calculation time.

**Add the appropriate effective address calculation time and the table search time.

***Number given is the maximum for a six-level table (FC lookup, a, b, c, and d levels with indirect level, all long descriptors).

11.8 Interrupt Latency

In real-time systems, the response time required for a processor to service an interrupt is a very important factor pertaining to overall system performance. Processors in the M68000 Family support asynchronous assertion of interrupts and begin processing them on subsequent instruction boundaries. The average interrupt latency is quite short, but the maximum latency is often critical because real-time interrupts cannot require servicing in less than the maximum interrupt latency. The maximum interrupt latency for the MC68030 alone is approximately 200 clock cycles (for the MOVEM.L ([d32,An],Xn,d32), D0-D7/A0-A7 instruction where the last data fetch is aborted with a bus error), but when the MMU is enabled, some operations can take several times longer to execute.

Interrupt latency in systems using the MMU is affected by the length of the main processor instructions, the address translation tree configuration, the number of translation tree searches required by the instructions, the access time of main memory, and the width of the data bus connecting the MC68030 to main memory. It is important to note that the address translation tree configuration is under software control and can strongly affect the system interrupt latency. The maximum interrupt latency for a given system configuration can be computed by adding the length of the longest main processor instruction to the time required for the maximum number of translation tree searches that the instruction could require. For the MC68030 microprocessor, one instruction of particular interest is a memory-to-memory move with memory indirect addressing for both the source and destination, with all of the code and data items crossing page boundaries. The assembler syntax for this instruction is:

```
MOVE.L (od,[bd,An,Rm]),(od,[bd,An,Rm])
```

This instruction can cause ten address translation tree searches: two for the instruction stream, two for the source indirect address, two for the destination indirect address, two for the operand fetch, and two for the destination write. System software can reduce the maximum number of translation searches by placing additional restrictions on generated code. For example, if the language translators in the system only generate long words aligned on long-word boundaries, the indirect address and operands can cause only one translation search each. This reduces the number of searches for the instruction to a maximum of six.

11.9 Bus Arbitration Latency

In a system that uses the MMU, the bus arbitration latency is affected by several factors. The MC68030 does not relinquish the physical bus while it is performing a read-modify-write operation. Since the address translation search is an extended read-modify-write operation, the no-cache-case latency is incurred by the longest address translation search required by the system.

Another bus arbitration delay occurs when a coprocessor or other device delays or fails to assert $\overline{DSACKx}$ or $\overline{STERM}$ signals to terminate a bus cycle. The maximum delay in this case is undefined; it depends on the length of the delay in asserting the signals.